

01

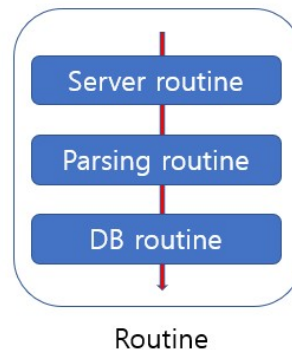
Overview



Coroutine

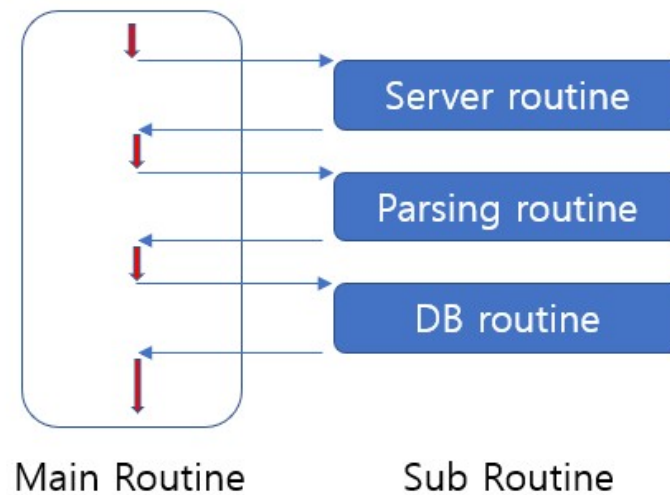
코루틴이란

- 코루틴(Coroutine)은 코틀린 에서 제공하는 비동기 프로그램을 위한 기법 혹은 라이브러리
- C++, Go, 파이썬, 코틀린등이 코루틴을 지원하는 대표적인 언어
- 코루틴(Co + routine)의 이름에서 Co 는 with 혹은 together
- 루틴이란 소프트웨어 개발에서 흔히 사용되는 용어로 하나의 업무처리 단위



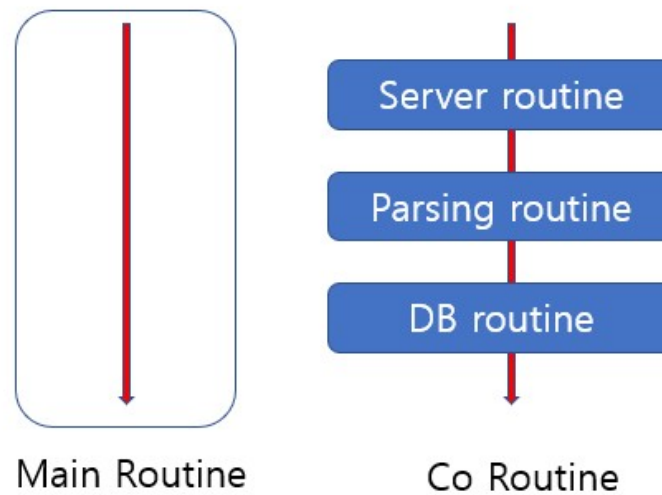
sub-routine vs co-routine

- sub-routine 은 단일 실행흐름에 의해 각 루틴들이 순차적으로 실행되는 구조
- 동기(Synchronous) 프로그래밍

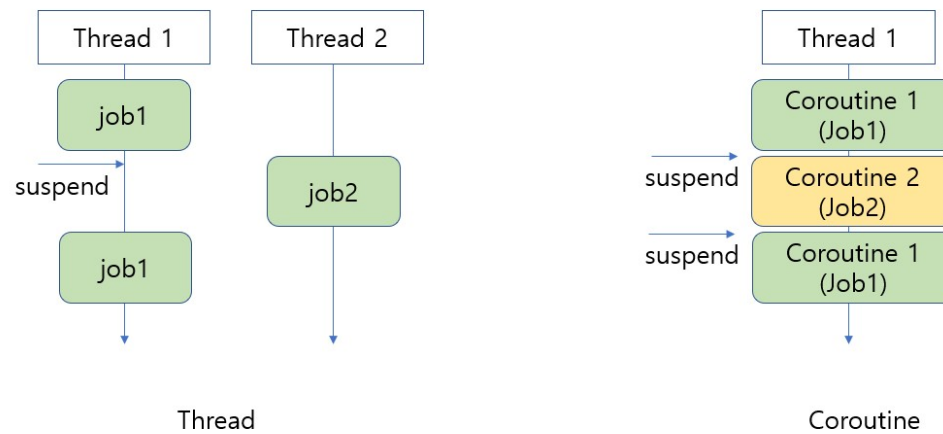


sub-routine vs co-routine

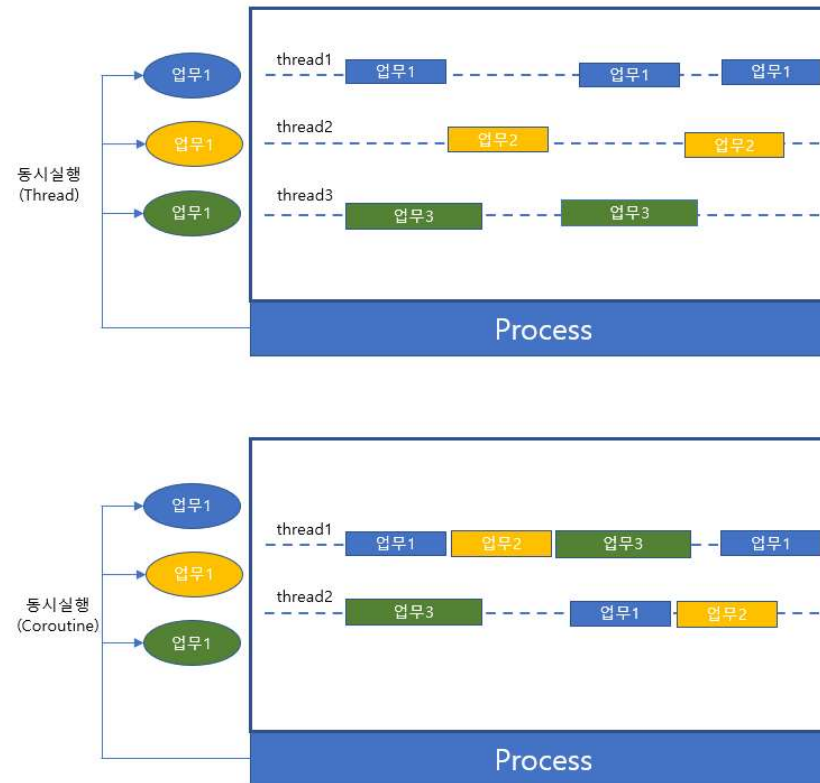
- Co – Routine 의 개념은 동시에 같이 실행되는 루틴을 의미
- 비동기(Asynchronous) 프로그래밍



Non-Blocking Lightweight Thread



Non-Blocking Lightweight Thread



라이브러리 등록

- Gradle

```
dependencies {  
    implementation("org.jetbrains.kotlin:kotlinx-coroutines-core:1.9.0")  
}
```

- Maven

```
<dependency>  
  <groupId>org.jetbrains.kotlin</groupId>  
  <artifactId>kotlinx-coroutines-core</artifactId>  
  <version>1.9.0</version>  
</dependency>
```

코루틴 핵심 개념 - Suspend Function

- suspend 예약어로 선언된 함수
- 중단 함수는 다른 중단 함수내에서만 호출 가능

suspend fun delay(timeMillis: Long)

- 함수를 실행시키는 스레드가 release 될 수 있는 함수

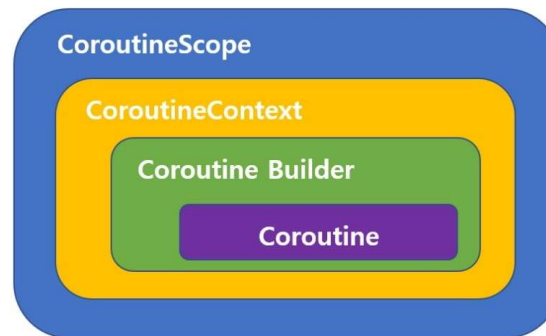
01

CoroutineScope

Coroutine

CoroutineScope

- 코루틴이 실행되는 영역
- 스코프를 선언하고 그 스코프에 CoroutineContext 를 지정
- 스코프에서 CoroutineContext 정보를 이용하여 코루틴이 만들어지고 실행



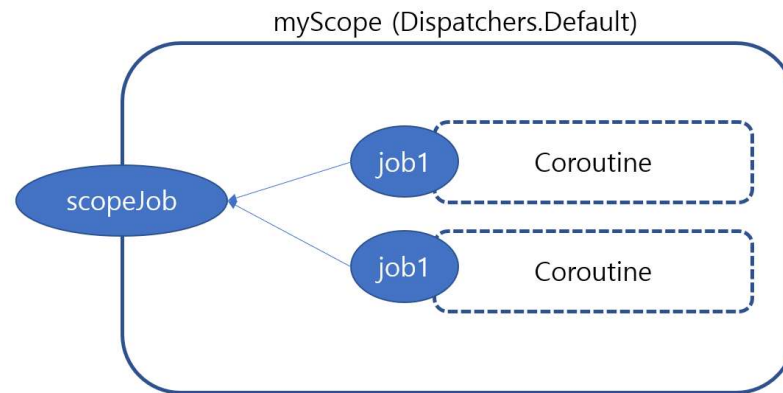
CoroutineScope

- 코루틴 빌더(launch, async 등)와 스코핑 함수(scoping function – coroutineScope, withContext 등)는 모두 스코프가 지정되어야 하며 그 스코프내에서 실행
- 스코프는 CoroutineScope 인터페이스를 구현한 클래스의 객체

```
interface CoroutineScope {  
    val coroutineContext: CoroutineContext  
}
```

CoroutineScope

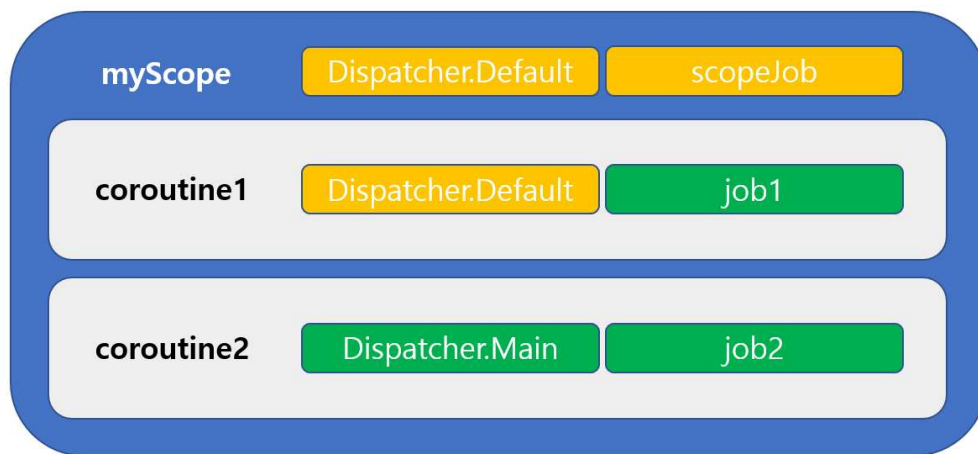
- 스코프는 코루틴을 구조화하기 위한 목적
- 스코프내에서 여러 코루틴을 실행시킴으로서 동일한 설정이 적용되게 할 수 있으며 한꺼번에 제어



CoroutineContext

- CoroutineContext 는 코루틴을 위한 설정 정보
- CoroutineContext 는 스코프에 선언하여 해당 스코프에서 실행되는 코루틴을 위한 공통의 설정도 가능하며 개별 코루틴에 설정하는 것도 가능
 - Job : 코루틴 생명주기 제어
 - CoroutineDispatcher : 코루틴이 실행될 스레드 지정
 - CoroutineName : 코루틴 이름
 - CoroutineExceptionHandler : 예외처리
- 여러가지 설정을 하고자 한다면 + 연산자를 이용

CoroutineContext



CoroutineDispatcher

- CoroutineContext 에 설정되는 정보로 코루틴이 어떤 스레드에서 동작해야 하는지를 명시
- CoroutineDispatcher 가 지정되지 않았다면 기본으로 Dispatchers.Default 가 적용
- 다른 코루틴(부모 코루틴) 내에서 실행되는 코루틴이라면(자식 코루틴) 부모 코루틴의 CoroutineContext 정보를 이용하게 됨으로 CoroutineDispatcher 를 지정하지 않았다면 부모 코루틴에서 사용한 CoroutineDispatcher 가 적용

Dispatchers

- Dispatchers.Main, Dispatchers.Default, Dispatchers.IO, Dispatchers.Unconfined 중 하나로 CoroutineDispatcher 를 설정
- Dispatchers.Main 은 앱을 실행시킨 스레드를 지칭
- 안드로이드 앱처럼 화면이 있는 경우 화면 출력을 하는 Main 스레드를 지칭

Dispatchers

- Dispatchers.Default 는 지속적으로 CPU 를 점유해 빠른 연산이 필요한 작업을 위해 설계
- Dispatchers.IO 는 네트워크, 데이터베이스 연동, 파일 작업등 대기시간이 있는 작업을 위해 설계
- Dispatchers.IO 의 경우 필요에 따라 스레드를 더 만들거나 줄이게 되며 최대 64개

Dispatchers

- `Dispatcher.Unconfined` 는 특정 스레드를 국한하지 않을 때 사용
- `Dispatcher.Unconfined` 로 지정하면 코루틴을 시작시키는 스레드에 의해 실행되지만 일시 중단함수 이후에 코루틴이 실행될 때는 가능한 모든 스레드에서 재개

다양한 CoroutineScope - GlobalScope

- GlobalScope 는 싱글톤으로 유지되는 CoroutineScope 타입의 객체
- 애플리케이션과 동일한 생명 주기
- GlobalScope 는 애플리케이션과 생존주기가 동일한 일종의 데몬 스레드처럼 동작하는 코루틴을 위해서만 제한적으로 사용해 주기를 권장

```
GlobalScope.launch {  
    repeat(5){  
        println("step1... : ${Thread.currentThread().name}")  
    }  
}
```

다양한 CoroutineScope – CoroutineScope()

- 매개변수에 지정되는 CoroutineContext 정보를 가지는 스코프를 만들어 주는 함수
- 함수로 간단하게 스코프를 정의하기 위해 사용

fun CoroutineScope(context: CoroutineContext): CoroutineScope

```
val scope = CoroutineScope(Dispatchers.Default)
scope.launch {
    println(".....")
}
```

다양한 CoroutineScope – MainScope()

- MainScope() 는 스코프를 만드는 함수
- CoroutineContext 은 SupervisorJob 과 Dispatchers.Main 로 고정
- 안드로이드 같은 UI 를 주 목적으로 하는 곳에서 간단하게 스코프를 정의해 사용하기 위한 목적으로 제공되는 함수

```
val scope1 = MainScope()
scope1.launch {
    //do something....
}
```

01

Coroutine Builder와 Job

Coroutine

runBlocking()

- runBlocking() 은 새로운 코루틴을 만들어 실행시키는 코루틴 빌더
- 코루틴이 모두 종료될 때까지 코루틴을 실행시킨 스레드는 대기상태
- main 함수 혹은 단위 테스트를 위한 함수에서 suspend 함수 혹은 코루틴 빌더를 사용하는 목적으로만 제한적으로 사용을 권장

```
runBlocking {  
    repeat(2){  
        delay(200)  
        println("coroutine $it")  
    }  
}
```

launch()

- 코루틴 빌더
- launch() 에 의해 만들어진 코루틴은 Non-Blocking 으로 실행
- launch() 함수는 Job 을 리턴

launch()

- launch() 함수의 매개변수에 CoroutineStart 를 지정
- 코루틴을 바로 실행 시켜야 하는지 아니면 필요한 순간에 실행시켜야 하는지를 제어
 - CoroutineStart.DEFAULT : 기본 값, 바로 시작, 시작되기 전에 취소하면 취소됨
 - CoroutineStart.ATOMIC : 바로 시작, 시작되기 전에 취소해도 시작됨.
 - CoroutineStart.LAZY : 어디선가 Job 의 start() 함수가 호출될 때 시작
 - CoroutineStart.UNDISPATCHED : 바로 시작, 시작되기 전에 취소해도 시작됨. 현재 스레드에서 시작되지만 일시중단 되었다가 다시 실행될 때 CoroutineContext 에 지정된 스레드에서 실행됨.

```
val job2 = launch(start = CoroutineStart.ATOMIC) {  
}
```

Job

- Job 은 코루틴에 의해 실행되는 업무를 지칭하는 객체
- launch() 함수의 리턴값으로 획득하는 방법과 Job() 함수로 획득하는 방법
- Job() 함수로 획득되는 객체는 CompletableJob 타입이며 Job 의 서브타입

```
val job2: CompletableJob = Job()
```

Job

- Job 객체의 함수를 이용해 코루틴에 의해 실행되는 업무를 취소시키거나 종료될 때 까지 대기
 - start() : 코루틴 시작
 - join() : 코루틴이 종료될 때까지 대기
 - cancel() : 코루틴 취소
 - cancelAndJoin() : 코루틴 취소, 취소될 때까지 대기
 - cancelChildren() : 하위 코루틴 취소

CompletableJob

- CompletableJob 은 Job 의 서브 타입
- complete() 와 completeExceptionally() 함수 두개가 추가
- complete() 함수를 이용해 Job 이 완료되었음을 선언
- completeExceptionally() 함수를 이용해 Job 에 예외를 발생시키고 완료되었음으로 선언

```
var isCompleted = job3.complete()
```

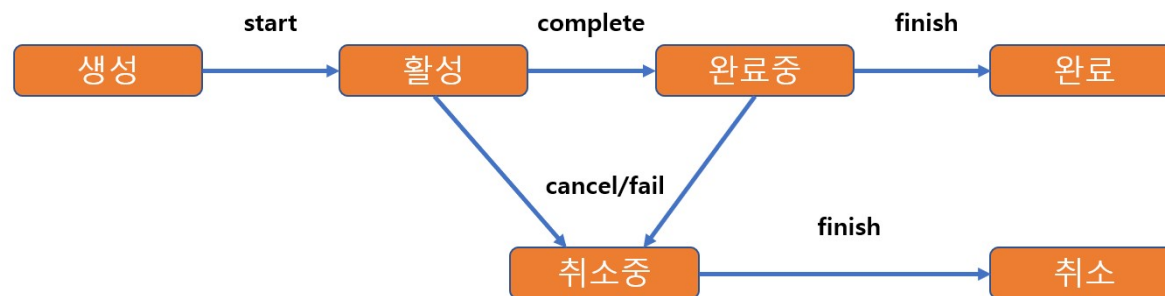
CompletableJob

- CompletableJob 을 사용하는 이유는 코루틴이 완료되었음을 명시적으로 선언
- complete() 함수로 실행중인 코루틴을 완료시킬 수 있지만 한번 완료된 CompletableJob 을 이용해 다른 코루틴을 구동시킬 수는 없다.

Job Lifecycle

- Job 이 가지는 상태는 생성상태, 활성상태, 완료중인 상태, 완료상태, 취소중인 상태, 취소상태
- 생성상태는 코루틴이 만들어졌지만 아직 실행되지 않은 상태
- 활성상태는 코루틴이 동작하고 있는 상태
- 취소중, 취소 상태는 활성상태의 코루틴을 어디선가 취소 시켰거나 코루틴에서 예외가 발생해서 종료되는 상태
- 완료중, 완료 상태는 코루틴의 업무가 정상적으로 처리되어 종료되는 상태

Job Lifecycle



Job Lifecycle

- 코루틴 내부 혹은 외부에서 코루틴이 어떤 상태에 있는지를 판단할 필요가 있는데 이를 위해 `isActive`, `isCompleted`, `isCancelled` 프로퍼티를 제공

상태	isActive	isCompleted	isCancelled
New	false	false	false
Active	true	false	false
Completing	true	false	false
Cancelling	false	false	true
Cancelled	false	true	true
Completed	false	true	false

Job cancel

- 실행중인 코루틴을 취소시키려면 Job 의 `cancel()`, `cancelAndJoin()` 등의 함수를 이용
- 함수가 호출되었다고 코루틴이 취소되지는 않는다.

```
job.cancelAndJoin()
```

Job cancel

- 코루틴 내부에서 자신이 cancel 된 것인지를 판단해서 cancel 되었다면 예외를 발생시켜 더 이상 실행되지 않게 해주어야 취소 됨.
- `delay()` 같은 함수에서 코루틴이 취소 된 것인지를 판단해 예외를 발생시켜 주었기 때문에 `cancel()` 에 의해 코루틴이 취소되었던 것.
- `kotlinx.coroutines` 에서 정의한 `suspend` 함수는 `delay()` 처럼 코루틴이 취소된 것인지를 판단해 예외를 발생
- 코루틴 내부에서 취소 명령이 내려진 것인지를 판단하면 되고 이를 위해 `Job` 에는 `isActive` 라는 프로퍼티를 제공

Job cancel

- `isActive` 로 판단해 예외를 발생시켜 코루틴이 취소될 때 마지막으로 처리해야할 코드
- `try – finally` 구문을 이용
- `finally` 부분에서 또다시 `cancel()` 에 의해 예외가 발생하는 코드가 작성된 경우 `finally` 부분이 정상적으로 실행되지 않는 문제가 발생할 수 있음

Job cancel

- 코루틴이 취소되어 예외가 발생 하더라도 정상적으로 실행되어야 하는 부분이 있다면

`withContext(NonCancellable) { }` 을 이용

```
withContext(NonCancellable) {  
    .....  
}
```

Job 예외처리

- Job 이 실행이 완료되거나 예외가 발생했을 경우 마지막으로 실행되어야 할 로직
- Job 의 `invokeOnCompletion()` 으로 함수를 등록하여 자동으로 실행

```
job.invokeOnCompletion { e ->  
    .....  
}
```

async 와 Deferred

- launch() 에 의해 실행된 값을 코루틴 외부에서 획득하는 방법을 제공하지 않음
- 코루틴을 실행시킨 곳에서 데이터를 획득해 이용해야 하는 경우 async() 이용
- async() 는 코루틴 빌더이며 async() 함수의 리턴 타입은 Deferred
- Deferred 는 Job 의 서브 타입임으로 Job 에서 제공하는 함수이외에 코루틴의 결과 데이터를 획득할 수 있는 await() 함수를 제공

01

Scope Function

Coroutine

스코프 함수

- Coroutine scope function(혹은 scoping function)
- 스코프를 만들고 Blocking 으로 실행되는 영역을 선언하기 위해서 제공
- 스코프를 편하게 사용하기 위한 목적
- 다양하게 제어되는 코루틴 영역을 명시하기 위한 목적
- `coroutineScope()`, `supervisorScope()`, `withContext()`, `withTimeout()`

coroutineScope()

- 대표적인 스코프 함수가 `coroutineScope()`
- `coroutineScope()` 로 만든 스코프는 외부(그 부분을 호출했던 코루틴) 스코프의 `coroutineContext` 를 상속
- `Job` 은 새로 정의되지만 `cancel` 함수를 제공하지 않는다. 직접 취소가 불가능하다.
- 상위 `coroutine` 이 취소되면서 `coroutineScope()` 가 취소되게 해야 한다.

```
coroutineScope {  
  
}
```

supervisorScope()

- 스코프에서 여러 개의 코루틴이 실행될 수 있으며 하나의 코루틴이 예외가 발생하게 되면 같은 스코프에서 동작하는 모든 코루틴이 종료
- 동일 스코프의 코루틴들이라고 하더라도 하나의 코루틴이 예외가 발생한 경우 나머지 코루틴이 정상적으로 실행되게 만들고 싶을 때 supervisorScope() 이용

```
suspend fun something2() = supervisorScope {  
    .....  
}
```

withContext()

- withContext() 는 스코프를 선언하면서 별도의 CoroutineContext 를 선언 가능.
- CoroutineContext 선언하여 외부 CoroutineContext 의 설정 중 일부를 withContext() 에서 교체

```
withContext(Dispatchers.IO){  
}
```

- NonCancellable 로 지정한 withContext() 영역은 어디선가 취소 명령이 발생했다고 하더라도 끝까지 정상 실행

```
withContext(NonCancellable){  
}
```

withTimeout()

- 코루틴에 의해 실행되는 영역 중 timeout 을 걸고 싶을 때 사용하는 스코프 함수

```
withTimeout(1300){  
}
```

01

Channel



Coroutine

Channel

- 코루틴이 실행되면서 발행하는 데이터를 반복적으로 구독하기 위한 방법은 Channel 과 Flow
- Channel 은 데이터를 주고 받기 위한 연결 통로
- Channel 자체가 코루틴이 아니며 단지 코루틴 간의 데이터를 주고 받는 방법
- 데이터를 발행하는 곳을 위한 SendChannel 인터페이스와 데이터를 구독 곳을 위한 ReceiveChannel 인터페이스가 있는데 이 둘을 상속받아 정의한 Channel 인터페이스 타입으로 채널을 이용

```
val channel = Channel<Int>()  
.....  
channel.send(0)  
-----  
channel.consumeEach { }
```

버퍼 이용 - capacity

- Channel 을 이용해 데이터를 발행 하고자 할 때 구독이 준비 안되었을 수도 있다.
- send() 는 어디선가 데이터를 구독 될 때까지 대기상태가 되는 것이고, receive() 는 어디선가 데이터가 발행될 때까지 대기상태가 된다.
- Channe() 함수의 기본 capacity 설정이 RENDEZVOUS 로 되어 있으며 RENDEZVOUS 는 capacity 를 0으로 설정



버퍼 이용 - capacity

- Channel() 함수를 이용하면서 capacity 를 조정하여 채널의 버퍼 사이즈를 조정
- 버퍼 사이즈는 숫자 값으로 지정하거나 아래의 상수 변수로 지정
 - BUFFERED : 버퍼 사이즈 64
 - CONFLATED : 버퍼 사이즈 1, 새로운 데이터가 이전 데이터 대체
 - RENDEZVOUS : 버퍼 사이즈 0
 - UNLIMITED : 수용량에 제한이 없다.

```
Channel<Int>(capacity = 2)
```


버퍼 이용 – onBufferOverflow

- 버퍼가 가득 찼을 때 어떻게 할 것인지 onBufferOverflow 로 제어
 - SUSPEND : 기본 값, send 대기
 - DROP_OLDEST : 가장 오래된 데이터 제거
 - DROP_LATEST : 가장 최신 데이터 제거

```
Channel<Int>( onBufferOverflow = BufferOverflow.DROP_OLDEST )
```

produce()

- produce() 는 채널을 이용하기 위해 만들어진 코루틴 빌더
- produce() 에 의해 만들어진 코루틴은 ProducerScope 에 담겨서 실행
- ProducerScope 는 CoroutineScope 를 구현한 것 뿐만 아니라 SendChannel 을 구현
- produce { } 에서는 별도의 Channel 객체를 준비하지 않아도 SendChannel 의 send() 함수를 이용해 데이터를 발행
- produce() 의 리턴 타입은 ReceiveChannel

```
val producer: ReceiveChannel<Int> = produce {  
    for (x in 1..5) {  
        delay(100)  
        send(x * x)  
    }  
}
```

actor()

- actor() 는 produce() 와 마찬가지로 채널을 이용하기 위한 코루틴 빌더
- actor() 는 CoroutineScope 와 ReceiveChannel 을 구현한 ActorScope 에서 코루틴이 실행
- actor() 의 리턴 타입은 SendChannel
- 외부에서 채널을 통해 데이터를 발행하면 그 데이터를 구독해서 동작하는 코루틴을 만들 때 사용

```
val sendChannel = actor<Int> {  
    consumeEach {  
  
    }  
}
```

01

Flow

Coroutine

Flow

- Flow 는 연속적으로 발생하는 데이터의 흐름
- Flow 를 만드는 방법은 여러가지가 있는데 가장 기본적인 방법이 `flow { }`
- `flow { }` 내에서 데이터 발행은 `emit()` 함수를 이용
- 발행한 데이터를 구독하는 것은 Flow 의 `collect()` 함수

```
fun something(): Flow<Int> = flow{  
    emit(0)  
}
```

Flow

- Flow 는 Cold Stream
- 어디선가 collect() 등의 함수로 데이터를 구독해야 Flow 는 실행
- collect() 는 Blocking 으로 실행
- 만약 데이터 구독을 Non-Blocking 으로 이용하고 싶다면 launch() 등으로 코루틴을 만들어 코루틴 내에서 Flow 를 구독

```
myFlow.collect {  
    println("collect 1 : $it")  
}
```

Flow Builder

- Flow Builder 는 Flow 를 만드는 함수
- flow() 이외에 flowOf(), asFlow(), channelFlow() 등이 제공

Flow Builder – flowOf()

- flowOf() 함수의 매개변수는 가변인수(vararg) 로 선언
- flowOf() 는 매개변수로 지정된 값들을 순차적으로 발행하는 Flow 를 만드는 함수

```
flowOf(1, "hello", true)
```

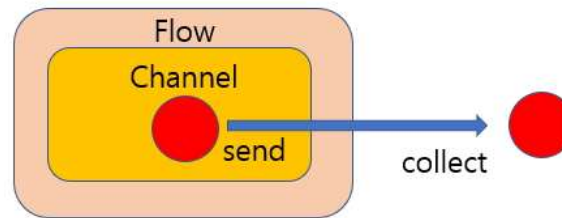

Flow Builder – asFlow()

- asFlow() 는 Array, Iterator 데이터를 Flow 로 만드는 함수

```
listOf(1,2,).asFlow()
```

Flow Builder - channelFlow()

- channelFlow() 는 Channel을 이용하는 Flow 를 만드는 함수
- channelFlow { } 에 지정되는 코드는 ProducerScope 에서 실행됨으로 { } 내에서 Channel의 send() 함수를 이용해 데이터를 발행 가능



- channelFlow() 이 사용되는 주 목적은 코루틴에서 발생하는 데이터를 Flow 로 처리하기 위해서 사용

MutableStateFlow

- Flow 의 하위 타입으로 MutableStateFlow 와 MutableSharedFlow 를 제공
- Flow 는 Cold Stream 이며 MutableStateFlow 와 MutableSharedFlow 은 Hot Stream
- Hot Stream 은 Cold Stream 의 반대 개념으로 어디선가 실행시키지 않아도 데이터 발행이 가능
- 여러 곳에서 데이터를 구독한다고 하더라도 처음부터 다시 실행되어 모든 데이터가 발행되는 것이 아니라 구독한 순간부터 발행되는 데이터를 이용하게 하는 스트림

MutableStateFlow

- MutableStateFlow 는 StateFlow 의 서브 타입이며 MutableStateFlow() 함수에 의해 만듦.
- MutableStateFlow 은 Flow 의 현재 상태를 표현하기 위해 사용
- 초기 상태 값을 위해 MutableStateFlow() 함수의 매개변수로 값을 지정
- MutableStateFlow 의 값 발행은 emit() 함수를 이용하거나 value 프로퍼티 값을 변경
- 값 획득은 collect() 함수를 이용

```
val stateFlow = MutableStateFlow(0)
```

MutableSharedFlow

- MutableSharedFlow 는 MutableSharedFlow() 함수에 의해 만들어 지는데 함수의 replay, extraBufferCapacity, onBufferOverflow 매개변수를 이용해 어떻게 값이 발행되어야 하는지를 설정 가능.
- replay 설정은 collect 하는 곳에서 구독할 수 있는 이전 데이터 개수
- 기본 값이 0 이며 0으로 설정 되어 있다면 이전 데이터를 구독하지 않는다는 의미

```
val sharedFlow = MutableSharedFlow<Int>(replay = 2)
```

MutableSharedFlow - extraBufferCapacity

- `extraBufferCapacity` 는 collect 하는 데이터 구독자가 있는 경우에만 의미가 있으며 replay 값이 1 이상인 경우에만 적용
- `extraBufferCapacity` 는 replay 에 의해 확보되는 버퍼 이외에 추가적으로 확보해야 하는 버퍼사이즈를 의미

MutableSharedFlow - onBufferOverflow

- onBufferOverflow 설정은 확보한 버퍼 사이즈에 데이터가 모두 담긴 상태에서 새로운 데이터를 발행해야 할 때 어떻게 할 것인지에 대한 설정
- 기본값은 SUSPEND 이며 이는 버퍼의 데이터가 구독자에 의해 구독될 때까지 emit() 은 대기상태
- DROP_OLDEST 로 설정하면 버퍼에 담긴 데이터 중 가장 오래된 데이터가 제거되면서 새로운 데이터를 발행
- DROP_LATEST 이면 최신데이터가 제거

MutableSharedFlow vs MutableStateFlow

- 초기값 유무
- 다양한 설정
- 동일한 값이 연속으로 발행되어야 하는 상황에서 다르게 동작
- MutableStateFlow 는 값 발행이 안되지만 MutableSharedFlow 은 발행

01

Flow Terminal operator

Coroutine

Terminal operator

- Flow 를 이용하기 위한 함수를 operator 라고 통칭
- Flow 의 데이터를 구독해 최종 이용하는 Terminal operator
- Flow 에 어떤 내용을 추가해 다시 Flow 를 만들어 주는 Intermediator operator
- Terminal operator 는 Flow 의 확장함수로 제공되는 함수들이며 Flow 데이터를 구독하는 역할을 하는 함수
- 가장 대표적인 Terminal operator 가 collect()

first()

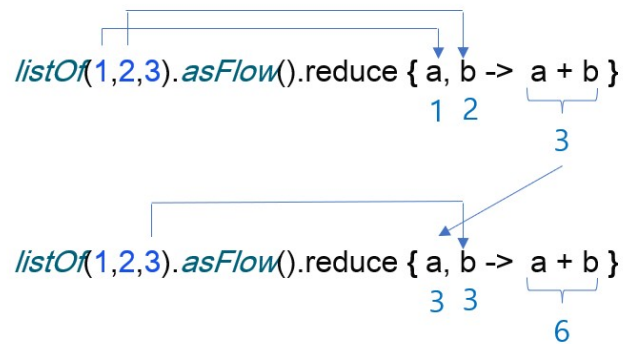
- first() 는 Flow 가 발행하는 첫번째 데이터를 구독하기 위한 operator
- 구독한 데이터를 획득만 하면 되면 first() 를 이용, 조건을 명시해 조건에 만족하는 첫번째 데이터를 획득하고 싶다면 first { } 를 이용
- first() 혹은 first { } 에 의해 획득되는 데이터가 없다면 예외가 발생
- firstOrNull() 혹은 firstOrNull { } 을 이용하면 획득되는 데이터가 없는 경우 null 을 리턴

single()

- single() 은 하나의 값만 구독하기 위한 Terminal operator
- single() 은 flow 가 종료될 때 까지 대기했다가 flow 가 종료되면 발행한 값을 리턴
- Flow 에서 발행하는 값이 없거나 하나 이상의 값을 발행하게 되면 예외가 발생

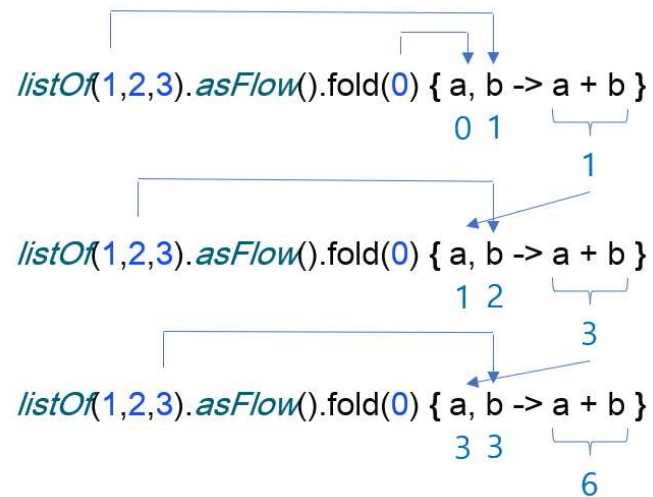
reduce(), fold()

- `reduce()` 와 `fold()` 는 Flow 발행에 의해 실행된 이전 결과 값을 새로운 데이터 발행에 의해 실행될 때 전달되어 참조할 수 있게 해주는 operator



reduce(), fold()

- fold() 는 reduce() 와 동일하게 동작하며 초기값 지정이 가능



toList(), toSet()

- toList(), toSet() 은 Flow 에 의해 발행되는 데이터를 MutableList, MutableSet 에 담아 반환시켜 주는 operator
- 모든 발행이 완료된 후 발행된 데이터를 모아 한꺼번에 처리하고자 할 때 유용

collectIndexed(), collectLatest()

- collectIndexed() 에 지정되는 함수는 매개변수를 두개 가지고 있으며 Flow 가 발행한 데이터 이외에 호출한 Index 값까지 전달
- collectLatest() 는 Flow 의 데이터 발행에 의해 collectLatest() 부분이 실행되는 도중 Flow 가 새로운 데이터를 발행하게 되면 collectLatest() 실행이 멈추고 새로운 데이터로 다시 실행

count()

- count() 는 Flow 에서 발행한 데이터의 개수를 반환시켜 주는 operator

launchIn()

- Flow 의 데이터 수신은 Blocking 으로 실행
- collect() 를 Non-Blocking 으로 실행하고 싶다면 코루틴을 만들고 그 코루틴에서 collect() 하게 해주어야 한다.
- launchIn() 은 코루틴에서 Flow 데이터를 이용되게 해주는 operator
- launchIn() 매개변수에 코루틴 스코프를 지정하면 자동으로 지정된 스코프내에서 코루틴이 실행
- launchIn() 으로 코루틴을 구동시킨 것임으로 리턴타입은 Job

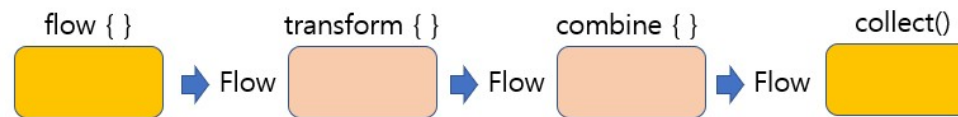
01

Flow Intermediate operator

Coroutine

Intermediate operator

- Intermediate operator 란 Flow 의 확장함수
- Flow 에 의해 발행되는 데이터를 다양하게 핸들링하는 방법을 제공해 주는 함수



map(), filter(), take()

- map() 은 Flow 에 의해 발행되는 데이터를 map { } 을 거쳐 구독되게 하기 위한 operator
- filter() 는 filter { } 에서 true 를 리턴한 데이터만 이용하게 하기 위한 operator
- take() 는 개수를 지정하여 그 개수까지의 데이터만 이용하기 위한 operator

flowOn()

- flowOn() 은 Flow 를 실행시킨 CoroutineContext 를 교체하기 위해서 사용되는 operator
- Flow 내부에서 이용되는 CoroutineContext 는 Flow 를 실행시킨 곳의 CoroutineContext 을 그대로 이용해야 하며 이를 Context Preservation 이라 한다.

stateIn(), shareIn()

- stateIn 과 shareIn 은 Flow 를 StateFlow 혹은 SharedFlow 로 바꾸기 위한 operator
- Flow 는 Cold Stream 이며 StateFlow, SharedFlow 는 Hot Stream
- stateIn, shareIn 을 이용해 Flow 의 Cold Stream 을 StateFlow 혹은 SharedFlow 을 이용하는 Hot Stream 으로 변경해 사용하고자 할 때 이용
- 하나의 Flow 에 의해 데이터가 발행되는데 이 데이터를 수신해야 하는 곳이 여러 곳인 경우 유용하게 이용

shareIn()

```
fun <T> Flow<T>.shareIn(  
    scope: CoroutineScope,  
    started: SharingStarted,  
    replay: Int = 0  
) : SharedFlow<T>
```

- replay 는 새로운 구독자가 발생했을 때 전달해야 하는 이전 발행 값의 개수
- 기본 값은 0이며 0보다 큰 수를 지정하면 그 수만큼의 버퍼가 설정되고 새로운 구독자를 위해 발행한 데이터를 버퍼에 저장
- scope 는 Hot Stream 을 실행하는 코루틴을 위한 스코프

shareIn()

- started 매개변수 값은 어떻게 데이터를 발행해야 하는지를 명시
 - Eagerly : 구독자가 없다고 하더라도 Hot Stream 이 만들어지면 바로 실행되어 데이터가 발행
 - Lazily : 첫번째 구독자가 존재해야 데이터를 발행하기 시작
 - WhileSubscribed() : 구독자가 존재하는 경우에만 데이터를 발행하며, 함수의 매개변수 설정 값으로 구독자가 사라진 후 어떻게 움직일 것인지를 지정
- WhileSubscribed() 함수의 매개변수에 설정되는 값
 - stopTimeoutMillis : 구독자가 모두 사라진 후 데이터 발행을 정지할 시간을 지정. 0 이면 바로 정지
 - replayExpirationMillis : 구독자가 모두 사라진 이후 replay 에 데이터를 유지해야 하는 시간

stateIn()

```
fun <T> Flow<T>.stateIn(  
    scope: CoroutineScope,  
    started: SharingStarted,  
    initialValue: T  
) : StateFlow<T>
```

- stateIn 도 shareIn 처럼 Cold Stream 을 Hot Stream 으로 변경하기 위한 operator 이며 반환 타입은 StateFlow
- 초기값을 initialValue 로 지정

transform()

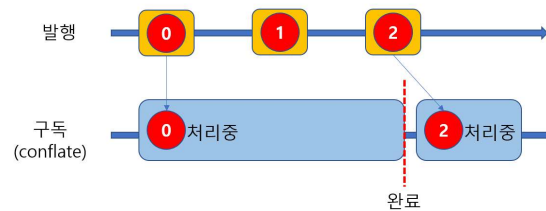
- transform() 은 Flow 에 의해 발행되는 데이터를 받아 새로운 데이터를 발행하고자 할 때 사용
- map(), filter(), take() 등의 다른 operator 와 다르게 transform { } 의 { } 에 지정한 코드는 FlowCollector 에서 실행
- transform { } 내에서 emit() 을 이용해 데이터 발행이 가능한 operator

buffer()

- 일반적으로 Flow 는 데이터 발행과 구독이 순차적으로 진행
- 데이터를 생산하는 속도가 데이터를 소비하는 속도보다 빠른 경우 미리 데이터를 발행해 버퍼에 담아 놓은 것이 효율적일 수 있는데 이때 사용하는 operator 가 buffer()
- buffer() 를 이용하면서 capacity 매개변수를 이용해 버퍼 사이즈를 지정
- onBufferOverflow 매개변수를 SUSPEND, DRAP_OLDEST, LATEST 중 하나를 지정하여 버퍼가 가득 찬 경우 새로 발행하는 데이터를 어떻게 해야할지 지정

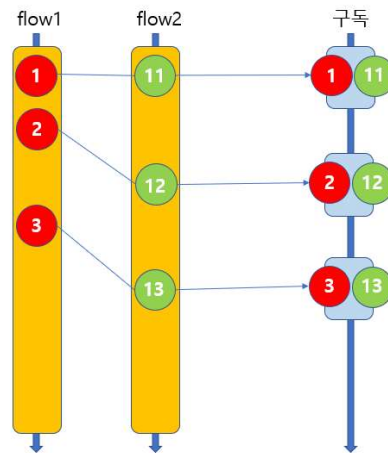
conflate()

- 데이터가 여러 번 발생한다고 하더라도 구독 쪽이 실행되는 시점의 최신데이터만 이용되게 하고 싶은 경우 사용하는 operator



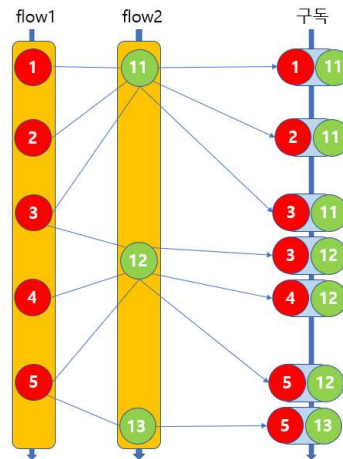
zip()

- zip() 은 두 flow 에서 발행하는 데이터를 같이 구독하기 위해 사용되는 operator
- zip() 에 의해 데이터가 발행되는 시점은 두 flow 의 데이터가 모두 새로 발행된 시점



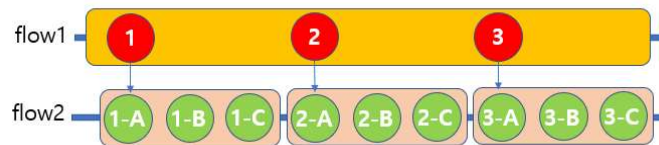
combine()

- combine() 도 zip() 과 마찬가지로 두 flow 의 발행 데이터를 같이 받기위해 사용
- combine() 은 어느 쪽 flow 든 새로운 데이터가 발행되기만 하면 combine() 에 의해 그 순간의 최신 데이터가 발행



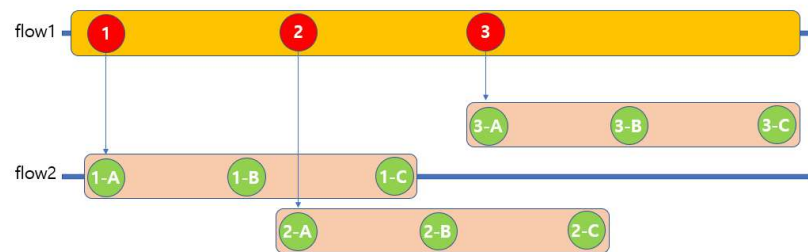
flatMapConcat()

- flatMapConcat() 와 flatMapMerge() 는 두 flow 를 연결해 하나의 flow에서 발행한 데이터에 의해 다른 flow 가 실행되게 할 때 사용
- flatMapConcat() 은 이중 루프와 흡사



flatMapConcat()

- flatMapMerge() 는 flatMapConcat() 과 목적은 동일합니다. 두 flow 를 연결하고 하나의 flow 에서 발행한 데이터에 의해 다른 flow 를 실행시키는 operator
- . flow1 과 flow2 가 있다고 가정하고 flow1에서 발행하는 데이터에 의해 flow2가 실행된다고 가정해보면 flatMapConcat() 은 순차적으로 flow2 가 실행되지만 flatMapMerge() 는 병렬적으로 flow2 가 실행



catch()

- catch() 는 Flow 에서 발생하는 예외를 처리하기 위한 operator
- catch() operator 영역은 FlowCollector 에서 동작하게 됨으로 Flow 에서 발생한 예외를 적절하게 처리한 후 다시 emit() 을 이용해 데이터를 발행 가능

onCompletion()

- onCompletion() 은 데이터 구독이 모두 완료된 후에 실행되는 operator
- Flow 에서 데이터가 발행이 완료되거나 아니면 예외가 발생해서 구독이 종료되는 경우에도 실행

02

Overview

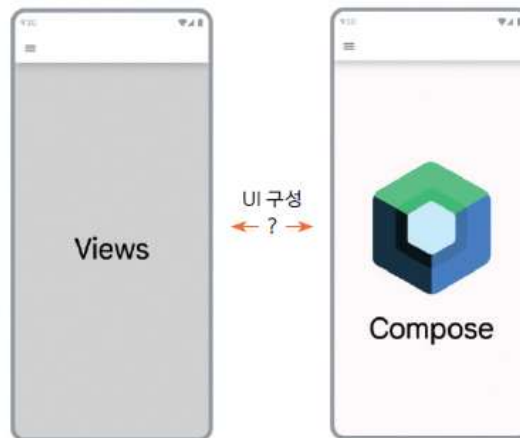


Compose

컴포즈 이해하기

컴포즈란?

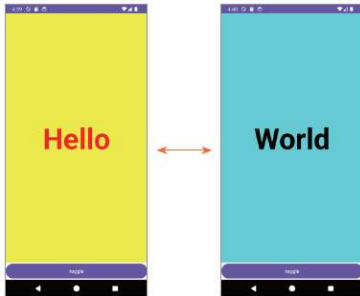
- 컴포즈compose란 제트팩에서 제공하는 기술로 앱의 화면을 구성하는 방법
- 뷰와 컴포즈는 상호 연동은 가능
- 컴포즈의 사용 목적은 선언형 UI 프로그래밍과 상태 관리



컴포즈 이해하기

■ 선언형 UI 프로그래밍이란?

- 선언형 UI 프로그래밍 Declarative UI Programming은 명령형 UI 프로그래밍 Imperative UI Programming과 반대되는 개념
- 명령형 UI 프로그래밍은 개발자 코드에서 UI 구성과 관련된 모든 코드를 작성하는 방식
- 명령형 UI로 화면을 구성하게 되면 화면 출력을 위한 개발자 코드가 길어질 수 밖에 없으며 개발자는 화면과 관련된 많은 API를 알고 있어야 합니다.



• 레이아웃 XML 파일(명령형 UI 프로그래밍으로 작성한 예)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:id="@+id/textView"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:text="Hello"
        android:textSize="80sp"
        android:gravity="center"
        android:textColor="#FF0000"
        android:background="#FFFF00"
        android:textStyle="bold"/>
```

• 버튼 클릭 시 화면 변경 처리(명령형 UI 프로그래밍으로 작성한 예)

```
binding.button.setOnClickListener {
    binding.textView.run {
        if(text == "Hello"){
            text = "World"
            setTextColor(Color.BLACK)
            setBackgroundColor(Color.CYAN)
        }else {
            text = "Hello"
            setTextColor(Color.RED)
            setBackgroundColor(Color.YELLOW)
        }
    }
}
```

컴포즈 이해하기

■ 선언형 UI 프로그래밍이란?

- 선언형 UI 프로그래밍은 화면에 어떻게 출력되어야 한다는 정보만 선언하는 방식

• 버튼 클릭 시 화면 변경 처리(선언형 UI 프로그래밍으로 작성한 예)

```
var textState by remember { mutableStateOf("Hello") }
var textColorState by remember { mutableStateOf(Color.Red) }
var textBackgroundColorState by remember { mutableStateOf(Color.Yellow) }
Column {
    Text(
        textState,
        fontSize = 40.sp,
        fontWeight = FontWeight.Bold,
        color = textColorState,
        textAlign = TextAlign.Center,
        modifier = Modifier
```

```
.fillMaxWidth()
.weight(1f)
.background(textBackgroundColorState)
.wrapContentHeight(align = Alignment.CenterVertically)
)
Button(modifier = Modifier.fillMaxWidth(), onClick = {
    if (textState == "Hello") {
        textState = "World"
        textColorState = Color.Black
        textBackgroundColorState = Color.Cyan
    } else {
        textState = "Hello"
        textColorState = Color.Red
        textBackgroundColorState = Color.Yellow
    }
}) {
    Text("toggle")
}
```

컴포즈 이해하기

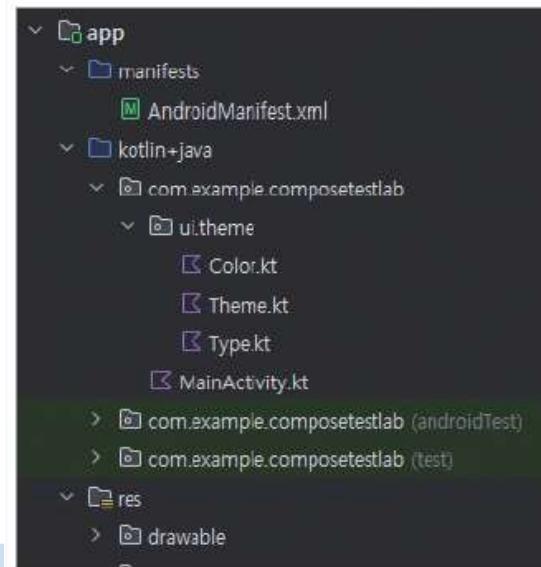
- 상태 관리란?

- 상태는 데이터입니다
- 상태는 화면에 출력되면서 다양한 이유(사용자 이벤트, 서버 네트워킹 등)에 의해 변경되고, 변경이 이뤄지면 화면 갱신이 되어야 하는 데이터를 의미
- 정보가 변경되었다는 것을 컴포즈에 인식시켜 컴포즈가 변경된 값으로 화면을 갱신

컴포즈 이해하기

컴포즈 프로젝트 구조

- 뷰를 사용하는 방식과 비교해 보면 전체적으로 디렉터리 파일 구성은 비슷하지만 컴포즈는 화면을 레이아웃 XML로 구성하지 않아 res 폴더 안에 layout 폴더가 만들어지지 않습니다.
- ui.theme 패키지에 앱의 테마를 선언하기 위한 코틀린 파일들이 자동으로 만들어집니다.



컴포즈 이해하기

build.gradle.kts(프로젝트 수준)

- 컴포즈는 코틀린으로 개발해야하므로 자동으로 코틀린과 관련된 플러그인이 추가

• 자동으로 추가된 코틀린 관련 플러그인 확인

```
plugins {  
    alias(libs.plugins.android.application) apply false  
    alias(libs.plugins.kotlin.android) apply false  
    alias(libs.plugins.kotlin.compose) apply false  
}
```

컴포즈 이해하기

build.gradle.kts(모듈 수준)

- 컴포즈를 사용한다는 선언이 android 영역에 자동으로 추가
- dependencies 부분에 컴포즈를 사용하기 위한 라이브러리가 자동으로 추가

• 컴포즈 사용 선언과 컴포즈 관련 라이브러리 추가

```
android {  
  
    buildFeatures {  
        compose = true  
    }  
}
```

```
}  
  
dependencies {  
  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.lifecycle.runtime.ktx)  
    implementation(libs.androidx.activity.compose)  
    implementation(platform(libs.androidx.compose.bom))  
    implementation(libs.androidx.ui)  
    implementation(libs.androidx.ui.graphics)  
    implementation(libs.androidx.ui.tooling.preview)  
    implementation(libs.androidx.material3)  
    (... 생략 ...)  
}
```

컴포즈 이해하기

MainActivity.kt

- 컴포즈를 이용하는 액티비티는 `ComponentActivity`를 상속
- 화면 출력을 `setContent()` 함수를 이용
- `setContent()`의 매개변수에 지정된 `AndroidLabTheme()`, `Surface()`, `Greeting()` 등은 컴포저블 `composable` 함수라고 부르며 화면을 구성하는 역할
- 컴포즈로 프로그램을 작성한다는 것은 컴포저블 함수를 만드는 것을 의미
- 컴포저블 함수는 `@Composable` 어노테이션으로 선언되는 함수

• 자동으로 생성되는 메인 액티비티 확인

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            AndroidLabTheme {  
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
                    Greeting(  
                        name = "Android",  
                        modifier = Modifier.padding(innerPadding)  
                    )  
                }  
            }  
        }  
    }  
}
```

• Greeting 컴포저블 함수 선언

```
@Composable  
fun Greeting(name: String, modifier: Modifier = Modifier) {  
    Text(  
        text = "Hello $name!",  
        modifier = modifier  
    )  
}
```

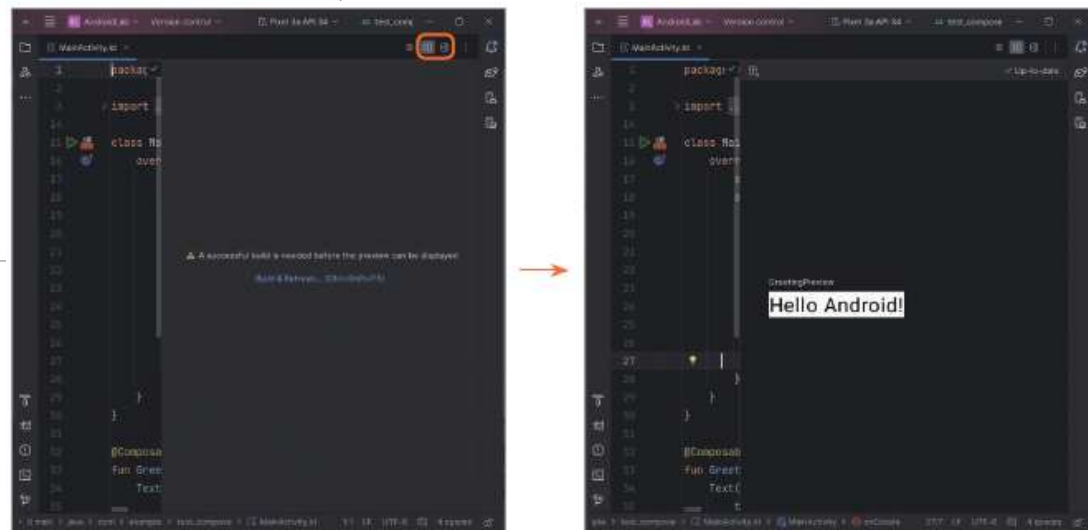
컴포즈 이해하기

MainActivity.kt

- GreetingPreview()는 @Preview 어노테이션으로 선언되며 실제 앱이 빌드되어 실행될 때의 화면 출력을 목적으로 하지 않고 안드로이드 스튜디오의 preview 화면에 출력될 내용을 표현하기 위한 컴포저블 함수

• GreetingPreview 컴포저블 함수 선언

```
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    AndroidLabTheme {
        Greeting("Android~")
    }
}
```



컴포즈 이해하기

컴포저블 함수 좀 더 알기

- 컴포즈에서 가장 중요한 부분이 컴포저블 함수
- 이미 제공되는 `Text()`, `Button()` 같은 컴포저블 함수를 이용해 개발자가 원하는 화면을 컴포저블 함수로 선언
- `@Composable` 어노테이션으로 선언
- 컴포저블 함수의 정보를 참조하고 이를 통해 화면을 구성
- 컴포저블 함수는 다른 컴포저블 함수 내에서만 호출
- 컴포저블 함수에서 일반 함수 호출이 가능
- 컴포저블 함수는 리턴값을 가질 수 있기는 하지만 리턴값이 화면 구성 정보로 사용되지 않았기 때문에 리턴값 자체가 의미가 없습니다.

02

상태 다루기

Compose

상태 다루기

상태란?

- 컴포저블 함수는 함수 내에 상태가 선언되어 어떻게 사용되는지에 따라 상태 컴포저블(Stateful Composable)과 비상태 컴포저블(Stateless Composable)로 구분

mutableStateOf 함수로 상태 선언

- 변수 자체를 상태라고 하지는 않습니다.

변수 선언의 예

```
var data1 = 0
```


상태 다루기

mutableStateOf 함수로 상태 선언

- 컴포저블 함수 내에서 상태 선언은 mutableStateOf 함수로 선언
- mutableStateOf()의 매개변숫값(여기서는 0)은 선언된 상태의 초기값
- 첫 번째는 상태를 이용하기 위한 변수이며 두 번째는 상태를 변경하기 위한 함수

• mutableStateOf 함수로 상태 선언

```
var (data2, setData2) = mutableStateOf(0)
fun changeData2() {
    setData2(Random.nextInt(0, 100))
}
```

- setter 함수를 호출하면 상태가 변경되고 화면이 재구성
- 상태값이 변경되고 컴포저블 함수가 다시 호출되기는 하지만 상태값은 항상 0
- 상태값을 변경해서 재구성(Recomposition)이 이뤄지지만 변경된 상태값이 유지되지 않았기 때문

상태 다루기

remember 함수를 사용해 상태값 유지

- 컴포저블이 재구성되어도 상태값을 유지되게 하려면 remember 함수를 같이 사용

• remember 함수를 사용한 예(by를 함께 사용한 예)

```
var data3 by remember {  
    mutableStateOf(0)  
}  
  
fun changeData3() {  
    data3 = Random.nextInt(0, 100)  
}
```

remember

- 컴포저블이 다시 컴пози션될 때 유실되면 안되는 데이터를 저장하기 위한 함수이다.
- remember 자체가 상태는 아니며 데이터를 저장하기 위한 용도이다.
- remember에 주로 저장하는 것이 상태인 것 뿐이지 상태가 아닌 다른 것을 저장해서 이용해도 된다.
- `val mutableState = remember { mutableStateOf(default) }`
- `val value by remember { mutableStateOf(default) }`
- `val (value, setValue) = remember { mutableStateOf(default) }`

rememberSaveable

rememberSaveable 함수

- 화면 회전과 같은 액티비티의 상태 변경에도 상태가 그대로 유지되게 하고 싶은 경우 rememberSaveable 함수를 이용해 상태를 선언

• rememberSaveable 함수로 상태 선언

```
var aData by remember {
    mutableStateOf(0)
}
var bData by rememberSaveable {
    mutableStateOf(0)
}
Column {
    Row {
        Text("remeber count : ${aData.toString()}", fontSize = 30.sp)
        Button(onClick = { aData++ }) {
            Text(text = "increment")
        }
    }
    Spacer(modifier = Modifier.height(10.dp))
    Row {
        Text("remeber count : ${bData.toString()}", fontSize = 30.sp)
        Button(onClick = { bData++ }) {
```

```
            Text(text = "increment")
        }
    }
}
```



02

Layout Composable

Compose

Modifier

모디파이어 사용하기

- 모디파이어 Modifier란 Modifier 객체 타입으로 표현되는 컴포저블의 설정 정보
- 컴포즈의 내장 컴포저블은 모두 매개변수로 modifier가 선언 되어 있다.
- 컴포저블을 설정하기 위한 모디파이어에는 100개 이상의 함수가 존재
- 패딩을 설정하기 위한 padding(), 컴포저블의 테두리를 설정하기 위한 border() 함수 등



Modifier

- then 함수를 이용해 다른 모디파이어를 조합할 수도 있습니다.
- 기본 설정을 위한 모디파이어 객체를 하나 준비하고 다른 모디파이어 객체에 새로운 설정을 추가하거나 기본 설정값을 변경

• then 함수로 모디파이어 추가

```
val modifier = Modifier
    .border(width = 10.dp, color = Color.Red)
    .padding(all = 30.dp)
    .background(Color.Yellow)

val secondModifier = Modifier.background(Color.Blue)
Text(
    text = "Hello World!",
    fontSize = 30.sp,
    modifier = modifier.then(secondModifier)
)
```

Modifier

background

```
Card(modifier = Modifier.background(Color.Blue)) {  
    Text("Hello")  
}
```

size/ width/ height

```
Modifier.background(Color.Blue)  
//          .size(200.dp, 100.dp)  
          .height(100.dp)  
          .width(200.dp)
```

fillMaxSize()/ fillMaxWidth() / fillMaxHeight()

Modifier

requiredSize

- 사이즈 지정 시 부모의 사이즈보다 자식의 사이즈가 크게 지정되면 적용되지 않을 수 있다.
- 이때 자식의 사이즈를 `requiredSize` 로 지정하면 부모에 제약되지 않는 자식의 사이즈를 지정할 수 있다.

```
Row(modifier = Modifier.size(100.dp, 100.dp)) {  
    Card(modifier = Modifier.requiredSize(200.dp,  
200.dp).background(Color.Red)) {  
        Text("hello")  
    }  
}
```

Modifier

padding

- Margin 은 따로 없다. padding 을 이용하거나 Spacer() 컴포저블을 이용해야 한다.

```
Modifier.  
//           .padding(24.dp)  
//           .padding(bottom = 24.dp, top = 24.dp, start = 24.dp, end = 24.dp)  
           .padding(vertical = 24.dp, horizontal = 24.dp)
```

clickable

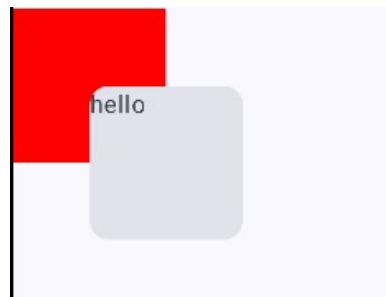
```
Card(modifier = Modifier.background(Color.Blue)  
    .size(200.dp, 100.dp)  
    .padding(24.dp)  
    .clickable {  
        Toast.makeText(context, "click", Toast.LENGTH_SHORT).show()  
    }) {  
    Text("Hello")  
}
```

Modifier

offset

- 이동 거리
- 양수, 음수 설정

```
Card(modifier = Modifier.size(100.dp, 100.dp)
    .background(Color.Red)
    .offset(50.dp, 50.dp)
) {
    Text("hello")
}
```



Modifier

weight

- Row / Column 의 modifier에만 적용

```
Row(modifier = Modifier.fillMaxWidth()) {  
    Card(modifier = Modifier.weight(2f).background(Color.Blue)) {  
        Text("hello")  
    }  
    Card(modifier = Modifier.weight(1f).background(Color.Red)) {  
        Text("hello")  
    }  
    Card(modifier = Modifier.weight(2f).background(Color.Green)) {  
        Text("hello")  
    }  
}
```



Modifier

clip

```
Box(  
  modifier = Modifier  
    .size(200.dp)  
    .clip(CircleShape)  
    .background(Color(0xFF4DB6AC))
```



Row, Column

Row와 Column 함수

- 컴포저블 여러 개를 등록하고 배치하기 위한 레이아웃
- Row, Column, Box, ConstraintLayout
- Row는 가로 방향 배 치에 관여하고 Column이 세로 방향 배치

• Row 컴포저블 함수를 활용해 가로 방향으로 문자열을 배치

```
@Composable
fun MainScreen() {
    Column {
        Row(modifier = Modifier.background(Color.Yellow)) {
            MyText(data = "A")
            MyText(data = "B")
            MyText(data = "C")
        }
    }
}
```

```
@Composable
fun MyText(data: String, modifier: Modifier = Modifier) {
    Text(
        data,
        fontSize = 50.sp,
        fontWeight = FontWeight.Bold,
        textAlign = TextAlign.Center,
        modifier = Modifier
            .border(width = 4.dp, color = Color.Black)
            .padding(10.dp)
            .then(modifier)
    )
}
```

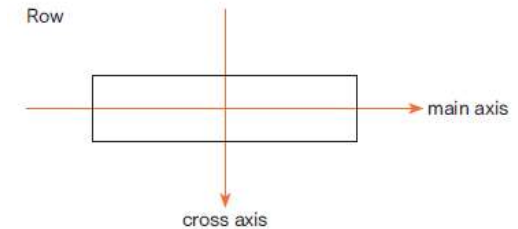
▶ 실행 결과



Row, Column

정렬

- 배치되는 방향을 주축(main axis), 반대 방향을 반대축(cross axis)이라 부르며 Row 의 주축은 가로 방향이고, 반대축은 세로 방향입니다.
- Column의 경우 주축은 세로 방향이고, 반대축은 가로 방향



Row, Column

- 정렬은 Alignment 혹은 Arrangement 객체를 이용하는데 주축에 대한 배치는 Arrangement를, 반대축의 배치는 Alignment를 이용
- Row, Column이 차지하는 화면 영역에 추가되는 컴포저블은 기본으로 좌측 상단에 정렬

• Alignment나 Arrangement를 사용하지 않은 경우

```
Row(modifier = Modifier
    .background(Color.Yellow)
    .fillMaxWidth()
    .height(300.dp)
) {
    MyText(data = "A")
    MyText(data = "B")
    MyText(data = "C")
}
```

▶ 실행 결과



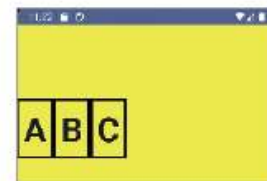
Row, Column

- Row에는 `verticalAlignment` 속성을 이용해 수직 방향의 정렬 위치를 설정합니다.
 - `Alignment.Top`: 상단 정렬
 - `Alignment.CenterVertically`: 세로 방향 중앙 정렬
 - `Alignment.Bottom`: 하단 정렬

• Row에 `verticalAlignment` 속성을 활용

```
Row(modifier = Modifier
    .background(Color.Yellow)
    .fillMaxWidth()
    .height(300.dp),
    verticalAlignment = Alignment.CenterVertically
) {
    MyText(data = "A")
    MyText(data = "B")
    MyText(data = "C")
}
```

▶ 실행 결과



Row, Column

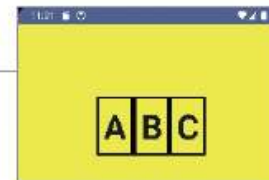
- Column에서는 `horizontalAlignment` 속성을 이용해 수평 방향의 정렬 위치를 설정
 - `Alignment.Start`: 왼쪽 정렬
 - `Alignment.CenterHorizontally`: 가로 방향 중앙 정렬
 - `Alignment.End`: 오른쪽 정렬
- Column에서 세로 방향 정렬을 하고자 한다면 `verticalArrangement`를 이용하여 `Arrangement.Top`, `Arrangement.Bottom` 등을 지정
 - `Arrangement.Start`: 가로 방향 왼쪽 정렬
 - `Arrangement.Center`: 중앙 정렬
 - `Arrangement.End`: 가로 방향 오른쪽 정렬
 - `Arrangement.Top`: 세로 방향 윗쪽 정렬
 - `Arrangement.Bottom`: 세로 방향 아래 정렬
 - `Arrangement.SpaceEvenly`: 컴포저블 사이에 균등한 여백을 주고 정렬
 - `Arrangement.SpaceBetween`: 컴포저블 사이를 균등한 여백으로 지정. 양쪽 끝 컴포저블과 레이아웃의 간격은 없음.
 - `Arrangement.SpaceAround`: 각 컴포저블 왼쪽, 오른쪽의 여백을 동일하게 지정해 정렬

Row, Column

- Row에 Alignment와 Arrangement 속성을 활용

```
Row(modifier = Modifier
    .background(Color.Yellow)
    .fillMaxWidth()
    .height(300.dp),
    verticalAlignment = Alignment.CenterVertically,
    horizontalArrangement = Arrangement.Center
) {
    (... 생략 ...)
}
```

▶ 실행 결과



Row, Column

• Arrangement.SpaceEvenly, Arrangement.SpaceBetween, Arrangement.SpaceAround 사용 예

```
Row(modifier = Modifier
    horizontalArrangement = Arrangement.SpaceEvenly
) {
    (... 생략 ...)
}
Row(modifier = Modifier
    horizontalArrangement = Arrangement.SpaceBetween
) {
    (... 생략 ...)
}
Row(modifier = Modifier
    horizontalArrangement = Arrangement.SpaceAround
) {
    (... 생략 ...)
}
```

▶ 실행 결과



Row, Column

스코프 모디파이어

- Row, Column에 추가되는 여러 컴포저블 함수가 동일 스코프
- 스코프 모디파이어는 Row, Column에 설정되는 것이 아니라 Row, Column에 추가되는 개별 컴포저블에 설정
- 자신이 포함된 스코프(Row 혹은 Column 영역) 내에 어떻게 배치되고, 어떤 크기를 가질지를 지정하기 위해 사용되는 모디파이어 속성

```
Row { this: RowScope  
    MyText(data = "A")  
    MyText(data = "B")  
    MyText(data = "C")  
}
```

Row, Column

- 대표적인 스코프 모디파이어로 weight 함수
- weight()는 스코프 내에서 개별 컴포저블의 사이즈를 지정하기 위해서 사용
- weight()는 다른 컴포저블의 weight()값을 같이 이용해 사이즈를 결정

• weight 값에 따른 사이즈 지정

```
Row(modifier = Modifier.background(Color.Yellow)) {  
    MyText(data = "A", modifier = Modifier.weight(1f))  
    MyText(data = "B", modifier = Modifier.weight(2f))  
    MyText(data = "C", modifier = Modifier.weight(1f))  
}
```

▶ 실행 결과



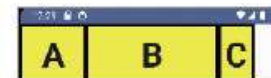
Row, Column

- fill 매개변수값을 true/false로 지정할 수 있는데 fill이 true면 가중치에 의해 계산된 크기만큼 화면에 차지하게 되고 false면 가중치에 의해 계산된 크기가 무시됩니다.

• fill에 의한 크기 적용 여부

```
Row(modifier = Modifier.background(Color.Yellow)) {  
    MyText(data = "A", modifier = Modifier.weight(weight = 1f, fill = true))  
    MyText(data = "B", modifier = Modifier.weight(weight = 2f, fill = true))  
    MyText(data = "C", modifier = Modifier.weight(weight = 1f, fill = false))  
}
```

▶ 실행 결과

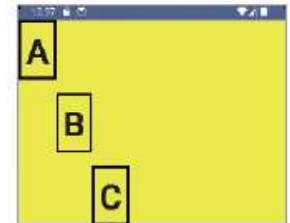


Row, Column

- align 함수로 지정되는 모디파이어는 개별 컴포저블에 설정해 동일 스코프 내에 컴포저블이 어느 위치에 배치될 것인지를 결정
- Alignment.CenterHorizontally, Alignment.Start, Alignment.End, Alignment.CenterVertically, Alignment.Top, Alignment.Bottom을 적용해 개별 컴포저블이 동일 스코프에서 어느 위치에 포함되어야 하는지를 지정

• align 함수를 이용해 모디파이어 지정

```
Row(modifier = Modifier.background(Color.Yellow).height(300.dp).fillMaxWidth()) {  
    MyText(data = "A", modifier = Modifier.align(Alignment.Top))  
    MyText(data = "B", modifier = Modifier.align(Alignment.CenterVertically))  
    MyText(data = "C", modifier = Modifier.align(Alignment.Bottom))  
}
```



Box

Box

- Box에 추가되는 컴포저블을 동일한 위치에 겹쳐서 출력

• Box 함수의 레이아웃 사용

```
Box {  
    MyText(data = "A")  
    MyText(data = "B")  
}
```

▶ 실행 결과

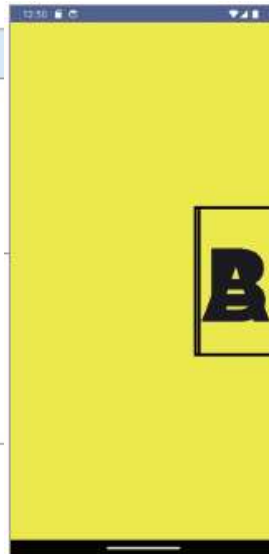


Box

- Box 함수에 추가되는 컴포저블의 정렬은 `contentAlignment`를 이용
- `Alignment.TopStart`, `Alignment.TopCenter`, `Alignment.TopEnd`, `Alignment.CenterStart`, `Alignment.Center`, `Alignment.CenterEnd`, `Alignment.BottomStart`, `Alignment.BottomCenter`, `Alignment.BottomEnd` 등을 사용해 정렬

• Box 함수에 추가되는 컴포저블 정렬

```
Box(  
  modifier = Modifier  
    .background(color = Color.Yellow),  
  contentAlignment = Alignment.CenterEnd  
) {  
  MyText(data = "A")  
  MyText(data = "B")  
}
```

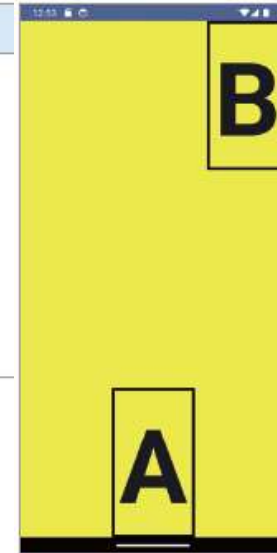


Box

- BoxScope 내에 추가되는 컴포저블에 align 모디파이어를 사용하면 개별 컴포저블의 위치를 지정할 수도 있습니다.

• align 모디파이어 사용

```
Box(  
  modifier = Modifier  
    .background(color = Color.Yellow)  
) {  
  MyText(data = "A", modifier = Modifier.align(Alignment.BottomCenter))  
  MyText(data = "B", modifier = Modifier.align(Alignment.TopEnd))  
}
```



ConstraintLayout

- ConstraintLayout은 다양한 조건으로 컴포저블을 배치하기 위한 레이아웃

• ConstraintLayout 사용 선언

```
implementation("androidx.constraintlayout:constraintlayout-compose:1.1.0")
```

- ConstraintLayout에는 부모 혹은 다른 컴포저블을 기준으로 컴포저블의 배치 조건을 명시
- 다른 컴포저블을 지정하기 위해서는 컴포저블을 식별할 수 있는 식별자가 있어야 하는데 이것을 참조라고 합니다.
- 참조 선언은 이와 같이 `createRef()`를 이용
- 여러 개의 참조를 선언하고 싶으면 `createRefs()`를 이용

• 참조 선언

```
val text1 = createRef()  
val (button1, button2) = createRefs()
```

ConstraintLayout

- ConstraintLayout : 참조 정보를 활용해 제약조건으로 배치하는 컴포저블
- ConstraintSet : 제약 조건을 별도로 관리하기 위한 객체, UI 구성과 제약 조건을 분리해서 개발하는 경우에 이용, 필수는 아님
- createRef(), createRefs() 와 createRefFor(), createRefsFor() 모두 참조를 선언하는 면에서는 동일하다.
- createRef(), createRefs() 의 경우 ConstraintLayoutScope 내에서 사용
- createRefFor(), createRefsFor() 은 ConstraintSetScope 에서 사용

ConstraintLayout

- 선언된 참조를 특정 컴포저블에 지정하려면 다음과 같이 `constrainAs` 모디파이어를 이용합니다.
- 위치에 관한 조건은 나의 위치, `linkTo`(대상의 위치) 형태로 작성합니다.

• `constrainAs`로 선언된 참조를 특정 컴포저블에 지정

```
ConstraintLayout(  
    Modifier  
        .size(150.dp, 150.dp)  
        .background(Color.Yellow)) {  
    // .....  
    MyText(data="Hello", modifier = Modifier.constrainAs(text1){  
        top.linkTo(parent.top, margin = 10.dp)  
        bottom.linkTo(parent.bottom, margin = 10.dp)  
    })  
}
```

▶ 실행 결과



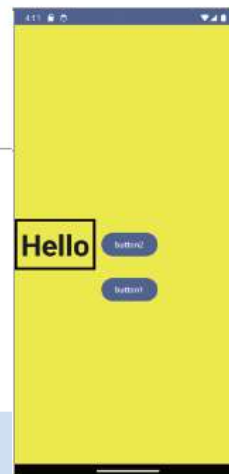
ConstraintLayout

- 다른 컴포저블을 기준으로 컴포저블의 위치를 지정하고 싶다면 기준이 되는 컴포저블에 미리 선언한 참조를 이용

• 선언된 참조를 사용해 다른 컴포저블 기준으로 위치 지정

```
Button(onClick = { /*TODO*/ }, modifier = Modifier.constrainAs(button1){
    top.linkTo(text1.bottom, margin=10.dp)
    start.linkTo(text1.end, margin = 10.dp)
}) {
    Text(text = "button1")
}
Button(onClick = { /*TODO*/ }, modifier = Modifier.constrainAs(button2){
    centerVerticallyTo(parent)
    centerHorizontallyTo(button1)
}) {
    Text(text = "button2")
}
```

▶ 실행 결과



ConstraintLayout - ConstraintSet

- ConstraintLayout 을 사용할 때 ConstraintLayout 과 제약조건 선언을 분리해서 작성
- ConstraintSet 을 이용해 제약조건을 선언하고 ConstraintSet 에 선언된 제약조건을 ConstraintLayout 에서 이용
- 코드의 간결함 혹은 동적 제약조건 지정등을 구현할 수 있게 된다.
- ConstraintSet 에서 참조를 선언할 때 createRefFor(), createRefsFor() 를 이용
- createRefFor(), createRefsFor() 의 매개변수에 참조의 별칭을 문자열로 지정하고 ConstraintLayout 에서는 그 별칭으로 참조를 이용

ConstraintLayout - ConstraintSet

```
private fun myConstraint(): ConstraintSet {  
    return ConstraintSet {  
        val button = createRefFor("button2")  
        val text = createRefFor("text2")  
  
        constrain(button) {  
            top.linkTo(parent.top, margin = 10.dp)  
            start.linkTo(parent.start, margin = 10.dp)  
        }  
        constrain(text) {  
            top.linkTo(button.bottom, margin = 10.dp)  
            start.linkTo(button.end, margin = 10.dp)  
        }  
    }  
}
```

```
val constraints1 = myConstraint()  
ConstraintLayout(  
    constraints1,  
    modifier = Modifier.background(Color.Green)  
) {  
    Button(onClick = {}, modifier = Modifier.layoutId("button2")) {  
        Text("Button1")  
    }  
    Text("Hello", modifier = Modifier.layoutId("text2"))  
}
```

ConstraintLayout - Guideline

- 화면에 부모를 기준으로 임의 선을 긋고 그 선을 기준으로 제약 조건을 지정하는 방법이다.
- 가로 가이드라인은 top, bottom 기준이며 세로 가이드라인은 start, end 기준이다.
 - `createGuidelineFromStart(0.1f)`
 - `createGuidelineFromEnd(16.dp)`
 - `createGuidelineFromTop(0.1f)`
 - `createGuidelineFromBottom(0.1f)`
- 가이드라인을 각 함수로 선언하면서 매개변수에 0.5f 처럼 값을 지정하면 % 개념이다. 즉 부모의 start 에서 50% 지점에 세로 가이드 라인 선을 선언하겠다는 의미이다.
- 100.dp 처럼 매개변수를 지정하면 부모의 top 에서 100dp 떨어진 지점에 가이드라인을 지정하겠다는 의미이다.

ConstraintLayout - Guideline

```
ConstraintLayout(  
    modifier = Modifier  
        .background(Color.Yellow)  
        .size(300.dp, 200.dp)  
) {  
    val guidelineStart = createGuidelineFromStart(0.5f)  
    val guidelineTop = createGuidelineFromTop(100.dp)  
  
    val (button1) = createRefs()  
  
    Button(onClick = {}, modifier = Modifier.constrainAs(button1) {  
        top.linkTo(guidelineTop, margin = 0.dp)  
        start.linkTo(guidelineStart, margin = 0.dp)  
    }) {  
        Text("Button1")  
    }  
}
```



ConstraintLayout - Barrier

- 가상의 선을 만들기 위함은 가이드 라인과 동일하다.
- 가이드 라인은 부모를 기준으로 % 혹은 스페이트 값을 이용해 가이드 라인을 지정함으로 정적이다.
- 배리어를 이용하면 다른 컴포지블을 기준으로 가상의 선을 구성할 수 있게 된다.
- 기준이 되는 컴포지블의 사이즈 변화에 따라 동적인 선이 만들어 질 수 있으며 여러 컴포지블을 묶어서 선을 만들 수도 있다.

```
val barrier1 = createEndBarrier(text1, text2)

Button(onClick = {}, modifier = Modifier.constrainAs(button) {
    top.linkTo(text2.bottom, margin = 10.dp)
    start.linkTo(barrier1, margin = 10.dp)
}) {
    Text("Button")
}
```

02

다양한 Composable

Compose

Text

- 리소스 문자열 출력

```
Text(stringResource(R.string.app_name))
```

- Color / fontSize / fontStyle / fontWeight

```
Text(  
    "HelloWorld",  
    color = Color.Red,  
    fontSize = 20.sp,  
    fontStyle = FontStyle.Italic,  
    fontWeight = FontWeight.Bold  
)
```

- style 속성으로도 가능하다.

```
Text(  
    "HelloWorld",  
    style = TextStyle(  
        color = Color.Red,  
        fontSize = 20.sp,  
        fontStyle = FontStyle.Italic,  
        fontWeight = FontWeight.Bold  
    )  
)
```

Text

- 그림자 효과

```
Text(  
    "HelloWorld",  
    style = TextStyle(  
        shadow = Shadow(color = Color.Blue, offset = Offset(5.0f, 5.0f),  
        blurRadius = 3f)  
    )  
)
```



HelloWorld

Text

- 하나의 Text 컴포저블에 여러 스타일 추가
- AnnotatedStyle 을 buildAnnotatedString 으로 만들어서 적용해야 한다.

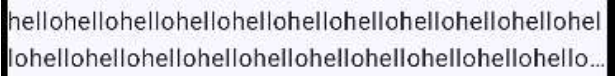
```
Text(  
    buildAnnotatedString {  
        withStyle(style = SpanStyle(color = Color.Blue)) {  
            append("H")  
        }  
        append("ello ")  
  
        withStyle(style = SpanStyle(fontWeight = FontWeight.Bold, color =  
Color.Red)) {  
            append("W")  
        }  
        append("orld")  
    }  
)
```

Hello **W**orld

Text

- maxLines, overflow
- 줄임 효과는 Ellipsis, Visible, Clip 중 하나 지정
- Ellipsis 이외에는 큰 의미가 없다.

```
Text(  
    "hello".repeat(50),  
    maxLines = 2,  
    overflow = TextOverflow.Ellipsis  
)
```

A visual representation of the text overflow effect. It shows two lines of the word "hello" repeated 50 times. The first line is truncated with an ellipsis (...). The second line is also truncated with an ellipsis (...). The text is enclosed in a black rectangular box.

hellohellohellohellohellohellohellohellohellohellohel
lohellohellohellohellohellohellohellohellohellohello...

Image

- 컴포즈에서 뷰에서 사용하던 Bitmap, Drawable 을 직접 출력할 수는 없다.
- 이미지가 화면에 출력되는 것은 어떤 API 에 의해 드로잉이 되어야 하는데 뷰에서 사용하던 Bitmap, Drawable 은 android.graphics 의 API 이다.
- 컴포즈에서는 이 API 를 사용하지 않고 androidx.compose.ui.graphics 에 있는 Painter 를 이용
- ImageBitmap, ImageVector 로 이미지 데이터를 지정하고 Painter 에 대입

Image

Painter

- 리소스 이미지

```
Image(  
    painter = painterResource(R.drawable.flower1),  
    contentDescription = "test image"  
)
```

- 비트맵 이미지

- asImageBitmap() 으로 Bitmap 을 ImageBitmap 으로 변환해서 이용

```
val bitmap: Bitmap = BitmapFactory.decodeResource(context.resources,  
    R.drawable.flower1)  
Image(  
    bitmap = bitmap.asImageBitmap(),  
    contentDescription = "test image"  
)
```

Image

아이콘 이미지

- <https://developer.android.com/reference/kotlin/androidx/compose/material/icons/package-summary>

```
implementation("androidx.compose.material:material-icons-extended:1.7.6")
```

```
Image(  
    imageVector = Icons.Filled.Call,  
    contentDescription = "test image"  
)
```

Image

네트워크 이미지 출력

- Coil 라이브러리 혹은 Glide 라이브러리 이용 권장
- Coil : Kotlin Coroutine 에서 지원하는 이미지 로드 라이브러리
- Glide : Google 에서 만든 이미지 로드 라이브러리

```
implementation(libs.coil)
//아래의 라이브러리 있어야 이미지 다운로드 된다.
implementation(libs.coil.network.okhttp)
```

```
AsyncImage (
    model = "https://~~~~",
    contentDescription = "test image",
    onError = {
    }
)
```

Image

ContentScale

- 이미지 크기와 출력 사이즈가 맞지 않았을 때 설정

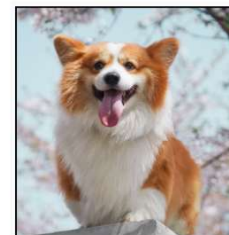
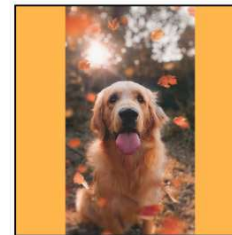
```
Image(  
    painter = painterResource(id = R.drawable.dog1),  
    contentDescription = "test",  
    contentScale = ContentScale.Fit,  
    modifier = imageModifier  
)
```

- ContentScale.Fit
 - 가로세로 비율 유지하면서 이미지 크기를 확대하거나 축소해서 컴포저블 경계에 맞춘다.



Image

- `ContentScale.Crop`
 - 이미지 가운데를 중심으로 컴포저블에 맞게 잘라서 출력
 - 확대 축소되지는 않는다.
- `ContentScale.FillHeight`
 - 비율 유지하면서 컴포저블의 높이에 맞추어서 확대/ 축소
- `ContentScale.FillWidth`
 - 비율 유지하면서 컴포저블의 너비에 맞추어서 확대/축소

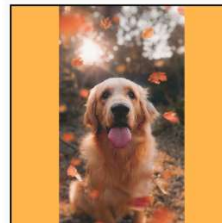


Image

- ContentScale.FillBounds
 - 가로 세로 컴포저블에 맞게 확대/ 축소, 비율이 유지되지 않는다.
- ContentScale.Inside
 - 컴포저블의 크기보다 이미지가 작으면 None
 - 크다면 fit 처럼 동작
 - 즉 큰 경우에만 비율 유지해 축소해서 출력



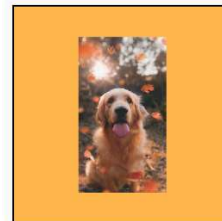
소스 이미지가 경계보다 큼



소스 이미지가 경계보다 큼



소스 이미지가 경계보다 작음



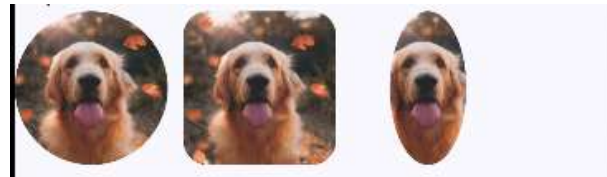
소스 이미지가 경계보다 작음



Image

- Clip

- 이미지 속성이라기 보다는 Modifier 속성
- 주로 이미지 출력에 많이 이용.



```
Image(  
    painter = painterResource(id = R.drawable.dog1),  
    contentDescription = "test",  
    contentScale = ContentScale.Crop,  
    modifier = Modifier  
        .size(100.dp)  
        .clip(CircleShape)  
)
```

- CircleShape, RoundedCornerShape 제공
- 만약 임의 도형대로 자르려면 Shape 상속받은 커스텀 클래스를 만들어 적용

Button

- Button

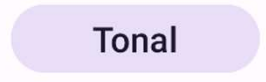
```
Button(onClick = { onClick() }) {  
    Text("Filled")  
}
```

A dark purple rounded rectangular button with the word "Filled" in white text.

Filled

- FilledTonalButton

```
FilledTonalButton(onClick = { onClick() }) {  
    Text("Tonal")  
}
```

A light purple rounded rectangular button with the word "Tonal" in dark purple text.

Tonal

Button

- OutlinedButton

Outlined

- ElevatedButton

Elevated

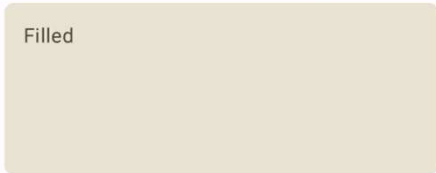
- TextButton

Text Button

Card

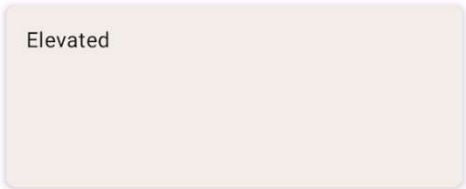
```
Card(  
  colors = CardDefaults.cardColors(  
    containerColor = MaterialTheme.colorScheme.surfaceVariant,  
  ),  
  modifier = Modifier  
    .size(width = 240.dp, height = 100.dp)  
) {  
  Text(  
    text = "Filled",  
    modifier = Modifier  
      .padding(16.dp),  
    textAlign = TextAlign.Center,  
  )  
}
```

Filled



■ ElevatedCard

Elevated



OutlinedCard

Outlined



Checkbox

- checked : 체크박스 값
- onCheckedChanged() : 이벤트 콜백
- 상태 값이 유지되어야 하고 그 값의 변경에 따른 re-composition 이 이루어져야 한다.

```
var checkState by remember {  
    mutableStateOf(true)  
}  
Checkbox(  
    checked = checkState,  
    onCheckedChange = {  
        checkState = it  
    }  
)
```

RadioButton

- selectableGroup
- 라디오 버튼을 이용하려면 selectableGroup 에 대해 먼저 이해해야 한다.
- selectableGroup 은 Modifier 속성으로 모든 컴포저블에 사용이 가능하다.
- 여러 컴포저블을 묶고, 그중 하나만 선택되게 하고자 할 때 이용된다.
- 각 항목의 selectable() 에서 해당 항목이 선택된 것인지, 클릭했을 때 이벤트를 처리를 명시

```
Row(modifier = Modifier.fillMaxWidth().selectableGroup().padding(10.dp)) {  
    arrayDatas.forEach {  
        Card(  
            .....  
            modifier = Modifier  
                .weight(1f)  
                .selectable(  
                    selected = (it == selected),  
                    onClick = { onChangeSelected(it) }  
                )  
        ) {  
            .....  
        }  
    }  
}
```

RadioButton

- RadioButton 은 여러 개중 하나만 선택됨을 목적으로 함으로 대부분 selectableGroup 으로 묶어서 사용이 된다.

```
Column(  
    modifier = Modifier  
        .selectableGroup()  
        .padding(16.dp)  
) {  
    options.forEach { option ->  
        Row(...) {  
            RadioButton(  
                selected = selectedOption == option,  
                onClick = { selectedOption = option },  
                modifier = Modifier.padding(end = 16.dp)  
            )  
            .....  
        }  
    }  
    .....  
}
```

Switch

- checked: 스위치 상태 값, Boolean
- onCheckedChangeListener : 이벤트 콜백
- enabled : 활성 상태
- colors
- thumbContent

```
Switch(  
    checked = switchState,  
    onCheckedChangeListener = { switchState = it },  
)
```


Slider

- value
- onValueChange
- valueRange
- color
- step : 단계

```
Slider(  
    value = sliderPosition,  
    onValueChange = { sliderPosition = it },  
    valueRange = 0f..100f  
)
```



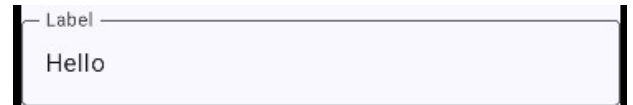
TextField / OutlinedTextField

- 상태에 의해 입력되는 값이 관리되어야 입력이 된다.

```
var text by remember { mutableStateOf("Hello") }  
TextField(  
    value = text,  
    onValueChange = { text = it},  
    label = { Text("입력") },  
    modifier = Modifier.fillMaxWidth()  
)
```



- OutlinedTextField



TextField / OutlinedTextField

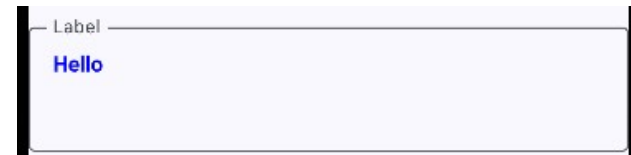
- singleLine / maxLines
- 기본은 한줄로 출력되다가 키보드에 의해 여러줄로 늘어난다.
 - singleLine : 한줄 고정
 - maxLines : 최대 라인 수
 - minLines : 최소 라인 수
- maxLines 와 minLines 를 동일한 수로 지정하여 고정 사이즈

```
OutlinedTextField(  
    value = text,  
    onValueChange = { text = it },  
    label = { Text("Label") },  
  
    maxLines = 3,  
    minLines = 3,  
    modifier = Modifier.fillMaxWidth()  
)
```

TextField / OutlinedTextField

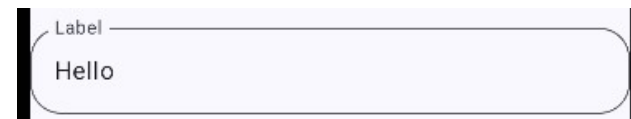
- `textStyle`
- Text 와 마찬가지로 입력되는 문자열의 스타일 지정 가능

```
textStyle = TextStyle(color = Color.Blue, fontWeight = FontWeight.Bold),
```



- `shape`
- OutlinedTextField 인 경우 테두리 모양 지정 가능
 - `CircleShape`
 - `RoundedCornerShape`
 - `RectangleShape`
 - `CutCornerShape`

```
shape = RoundedCornerShape(20.dp),
```



TextField / OutlinedTextField

- leadingIcon / trailingIcon

```
leadingIcon = {  
    Icons.Outlined.Lock  
},  
trailingIcon = trailingIconView
```



- keyboardType

- KeyboardType.Email
- KeyboardType.Number
- KeyboardType.NumberPassword
- KeyboardType.Password
- KeyboardType.Phone
- KeyboardType.Text
- KeyboardType.Uri

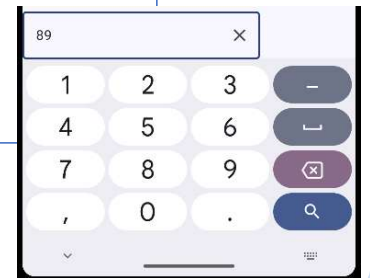


```
keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number)
```

TextField / OutlinedTextField

- imeAction / keyboardActions
- imeAction 은 우측 하단 키를 어떤 키로 사용할 것인가에 대한 설정
- keyboardActions 는 우측 하단 키를 클릭 했을 때의 이벤트 처리

```
keyboardOptions = KeyboardOptions(  
    keyboardType = TextInputType.Number,  
    imeAction = ImeAction.Search  
) ,  
keyboardActions = KeyboardActions(  
    onSearch = {  
        Toast.makeText(context, "search click", Toast.LENGTH_SHORT).show()  
    },  
    onDone = {  
        Toast.makeText(context, "done click", Toast.LENGTH_SHORT).show()  
    })
```



List

리스트

- 목록 화면을 구성하기 위해 LazyColumn, LazyRow, LazyVerticalGrid와 같은 함수를 제공합니다.
- 'lazy'가 붙는 이유는 '늦은 로딩'을 지원하는 컴포저블이기 때문입니다.
- LazyColumn을 이용하면 다음과 같이 항목을 세로 방향으로 나열
- items() 를 이용해 각 항목이 어떻게 구성되는지를 명시

• LazyColumn으로 세로 방향으로 항목 나열

```
LazyColumn {  
    val datas = listOf<String>("one", "two", "three", "four")  
    items(datas) { item ->  
        Text("$item ")  
    }  
}
```

▶ 실행 결과



List

- items() 대신 itemsIndexed()를 사용할 수 있는데 이렇게 되면 각 항목별로 index까지 출력

• itemsIndex 함수로 항목별 인덱스 출력

```
LazyColumn {  
    val datas = listOf<String>("one", "two", "three", "four")  
    itemsIndexed(datas){index, item ->  
        Text("$index $item")  
    }  
}
```

▶ 실행 결과

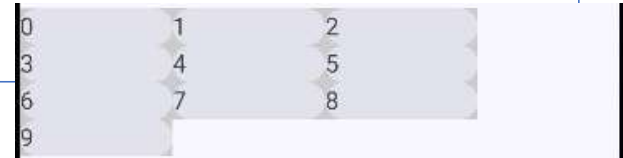


0 one
1 two
2 three
3 four

LazyVerticalGrid, LazyHorizontalGrid

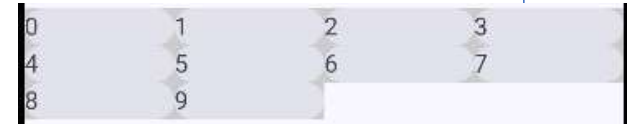
- LazyVerticalGrid 에서는 columns 로, LazyHorizontalGrid 에서는 rows 로 한 줄에 몇 개의 item 을 배치할 것인지를 결정
- columns 와 rows 의 값은 GridCells 이어야 하는데 이 값을 주는 방법이 여러가지가 있다.
 - GridCells.FixedSize(100.dp) : 각 셀이 차지할 사이즈 지정
 - GridCells.Fixed(4) : 한 줄에 셀 개수 지정
 - GridCells.Adaptive() : 동적 셀 개수 지정

```
LazyVerticalGrid(columns = GridCells.FixedSize(100.dp)) {  
    items(10) { item ->  
        Card(modifier = Modifier.background(Color.LightGray)) {  
            Text("$item")  
        }  
    }  
}
```



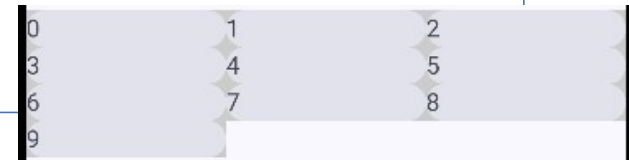
LazyVerticalGrid, LazyHorizontalGrid

```
LazyVerticalGrid(columns = GridCells.Fixed(4) ) {  
    items(10){ item ->  
        Card(modifier = Modifier.background(Color.LightGray)) {  
            Text("$item")  
        }  
    }  
}
```



0	1	2	3
4	5	6	7
8	9		

```
LazyVerticalGrid(columns = GridCells.Adaptive(minSize = 100.dp) ) {  
    items(10){ item ->  
        Card(modifier = Modifier.background(Color.LightGray)) {  
            Text("$item")  
        }  
    }  
}
```



0	1	2	
3	4	5	
6	7	8	
9			

Scaffold

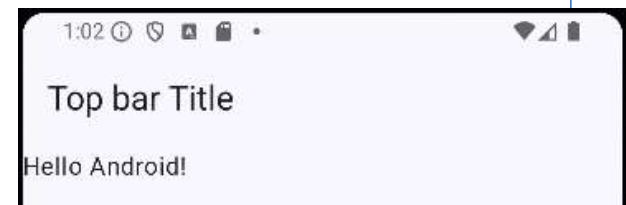
- 앱의 구조를 빠르게 조합하기 위한 컴포저블
- Material2 와 Material3 에 차이가 있다. Material3를 사용할 것을 권장
 - topBar
 - bottomBar
 - snackbarHost
 - floatingActionButton
 - floatingActionButtonPosition
 - contentColor
 - content
 - containerColor

AppBar

- 화면 상단 구성

- title
- colors
- navigationIcon
- actions

```
Scaffold(  
  modifier = Modifier.fillMaxSize(),  
  topBar = {  
    AppBar(  
      title = {  
        Text("Top bar Title")  
      }  
    )  
  }  
)
```



AppBar

- appBar 에 색상 지정하기
- colors 속성을 이용해 AppBar 의 색상 및 AppBar 콘텐츠의 색상 지정

```
AppBar(  
  title = {  
    Text("Top bar Title")  
  },  
  colors = appBarColors(  
    containerColor = Color.Red,  
    titleTextColor = Color.White  
  )  
)
```

TopAppBar

navigationIcon

```
navigationIcon = {  
    IconButton(onClick = {}) {  
        Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "")  
    }  
}
```



actions

```
actions = {  
    IconButton(onClick = {}) {  
        Icon(Icons.Filled.Search, contentDescription = "")  
    }  
    .....  
}
```



AppBar

AppBar 종류

- TopAppBar
- CenterAlignedTopAppBar : title 이 가운데 정렬
- MediumTopAppBar : title 이 아이콘 아랫줄에 정렬
- LargeTopAppBar : title 과 아이콘 사이 공간이 있는 큰 사이즈



LargeTopAppBar

TopAppBar

앱바 접히기

- 아랫 부분이 스크롤 될 때 앱바가 같이 스크롤 되려면 behavior 가 선언되어야 한다.

```
val scrollBehavior =  
TopAppBarDefaults.exitUntilCollapsedScrollBehavior(rememberTopAppBarState())
```

- 그런후에 Scaffold Modifier 에 선언된 behavior 를 등록해 이 behavior 에 화면의 스크롤 정보가전달되게 해주어야 한다.

```
Scaffold(  
    modifier =  
    Modifier.fillMaxSize().nestedScroll(scrollBehavior.nestedScrollConnection),
```

- 또한 AppBar 에 behavior 를 등록해 이 behavior 의 정보에 의해 앱바가 접히게 설정해 주어야 한다.

```
LargeTopAppBar(  
    .....  
    scrollBehavior = scrollBehavior  
)
```


AppBar

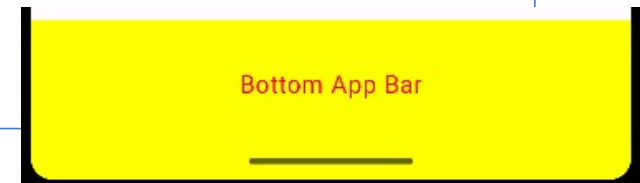
앱바 접히기

- Behavior 를 만들 때 여러 함수가 제공되며 어느 함수를 이용하느냐에 따라 스크롤이 다르게 동작한다.
 - `exitUntilCollapsedScrollBehavior()` : 거꾸로 스크롤 할 때 맨 마지막에 앱바가 펼쳐진다.
 - `enterAlwaysScrollBehavior()` : 거꾸로 스크롤 할 때 가장 먼저 앱바가 펼쳐진다.
 - `pinnedScrollBehavior` : 스크롤에 반응하지 않는다.

BottomAppBar

- bottomBar 는 Scaffold 영역의 하단에 나오며 BottomAppBar 이외에 다른 컴포저블을 사용해도 된다.
- 대표적으로 많이 사용되는 것이 BottomAppBar 와 NavigationBar 이다.

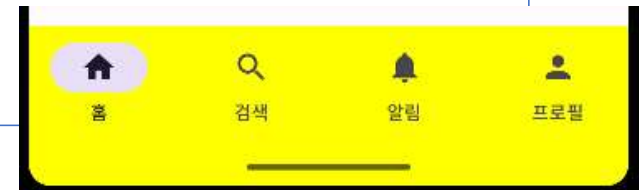
```
bottomBar = {  
    BottomAppBar(  
        containerColor = MaterialTheme.colorScheme.primaryContainer,  
        contentColor = MaterialTheme.colorScheme.primary  
    ) {  
        Text(  
            modifier = Modifier.fillMaxWidth(),  
            textAlign = TextAlign.Center,  
            text = "Bottom App Bar"  
        )  
    }  
}
```



BottomAppBar

- NavigationBar 를 이용해서 탭 화면 효과를 낸다.

```
NavigationBar(  
  containerColor = MaterialTheme.colorScheme.primaryContainer,  
  contentColor = MaterialTheme.colorScheme.onPrimaryContainer,  
) {  
  items.forEachIndexed { index, item ->  
    NavigationBarItem(  
      icon = { Icon(Icons[index], contentDescription = item) },  
      label = { Text(item) },  
      selected = selectedItem == index,  
      onClick = { selectedItem = index }  
    )  
  }  
}
```



FloatingActionButton

- FloatingActionButton 은 Scaffold 의 구성요소로 추가해도 되고, 독립적으로 사용해도 된다.
- FAB 를 구성하는 것은 Text 등 다양한 것으로 구성이 가능하지만 일반적으로 Icon

```
SmallFloatingActionButton(  
    onClick = {},  
    containerColor = MaterialTheme.colorScheme.secondaryContainer,  
    contentColor = MaterialTheme.colorScheme.secondary  
    ) {  
    Icon(Icons.Filled.Add, "ADD")  
}
```

- Scaffold 구성요소로 추가

```
floatingActionButton = {  
    FloatingActionButton(onClick = { }) {  
        Icon(Icons.Default.Add, contentDescription = "Add")  
    }  
}
```

02

Navigation



Compose

NavHostController

내비게이션

- 컴포저블을 교체하면서 화면 전환
- 화면 전환을 하기 위해서는 navigation 라이브러리를 이용

• navigation 라이브러리 추가

```
implementation("androidx.navigation:navigation-compose:2.8.4")
```

NavHostController

- navigation 라이브러리의 핵심 클래스는 NavHost와 NavHostConstroller입니다.
- NavHost는 화면 단위의 컴포저블을 등록하고 화면을 스택 구조로 관리하는 역할
- NavHostController 는 실제 필요한 경우 화면 전환을 하기 위한 함수를 제공

• navigation 라이브러리 추가

```
@Composable
fun MainScreen() {
    //NavController를 먼저 선언하고 NavController로 화면을 등록하는 NavHost를 만든다.
    val navController = rememberNavController()
    NavHost(navController = navController, startDestination = "home" ){
        composable("home"){ HomeScreen(navController) }
        composable("one"){ OneScreen(navController) }
        composable("two"){ TwoScreen(navController) }
    }
}
```

NavHostController

- NavHostController(androidx.navigation.compose) 는 NavController(androidx.navigation) 의 하위 타입이다.
- Navigation 을 위한 대부분의 구현은 NavController 에 구현되어 있으며 NavHostController 는 아래의 4개 함수가 추가된 형태이다.
- enableOnBackPressed() : 시스템 뒤로가기 버튼에 반응할 것인지 설정
- setLifecycleOwner() : LifecycleOwner 를 지정
- setOnBackPressedDispatcher() : 시스템 뒤로가기 버튼 이벤트를 처리하기 위한 OnBackPressedDispatcher 설정
- setViewModelStore() : 탐색 대상간 ViewModel 관리

NavHostController

- 컴포즈에서는 두가지 종류의 NavHost 이용
- `androidx.navigation.NavHost`
- `androidx.navigation.compose.NavHost`
- 컴포즈에서 두 NavHost 모두 이용이 가능하지만 `androidx.navigation.compose.NavHost` 를 이용해야 `navigation()` 에 의한 설정이 가능하며 이를 이용해 컴포저블간 뷰모델을 공유시킬 수 있다.
- `compose.NavHost` 를 이용하려면 `NavHostController` 이어야 한다.
- 컴포즈에서는 `NavHostController` 를 이용하는 것이 좋다.

NavHostController

- NavHost에 등록된 컴포저블에서 화면 전환 명령을 내릴 때는 NavHost에 등록된 NavController의 navigate 함수를 호출하면서 매개변수로 이동하고자 하는 화면의 이름을 명시

• 화면 전환을 위한 navigate 함수 호출

```
Button(onClick = { navController.navigate("one") }) {  
    Text("Go One")  
}
```

NavHostController

이전 화면으로 이동

- 안드로이드의 '뒤로 가기' 버튼에 의한 이동은 별도로 처리하지 않아도 이전 화면 컴포저블로 잘 이동됩니다.
- 프로그램적으로 이전 화면으로 전 환하고 싶다면 NavController의 popBackStack 함수를 호출합니다.

• 이전 화면으로의 전환을 위한 popBackStack 함수 호출

```
Button(onClick = { navController.popBackStack() }) {  
    Text("Go Back")  
}
```

NavHostController

- 이전 컴포저블 화면이 아닌 특정 컴포저블 화면으로 이동하고 싶다면 `navigate()` 함수를 호출하고 `popUpTo()` 함수로 옵션을 지정하면 됩니다.

• 특정 컴포저블 화면으로 이동하기 위한 `popUpTo` 함수 호출

```
Button(onClick = {  
    navController.navigate("onesub"){  
        popUpTo("home")  
    }  
}) {  
    Text("Go OneSub")  
}
```

NavHostController

데이터 전달

- 이름에 전달해야 하는 데이터를 같이 명시해서 등록할 수 있습니다.

• composable 함수에 데이터 추가

```
composable("one/{userId}/{no}", arguments = listOf(navArgument("userId"){  
    type = NavType.StringType  
}, navArgument("no"){  
    type = NavType.IntType  
})){ navBackStackEntry ->  
    OneScreen(  
        navController,  
        navBackStackEntry.arguments?.getString("userId"),  
        navBackStackEntry.arguments?.getInt("no")  
    )  
}
```

• navigate 함수를 호출해 전달할 데이터 명시

```
Button(onClick = { navController.navigate("one/kkang/20") }) {  
    Text("Go One")  
}
```

NavController

- 이전 화면으로 되돌아갈 때 현재 화면에서의 결과 데이터를 전달하겠다면 popBackStack 함수 실행 전에 다음과 같이 set 함수를 이용해 결과 데이터를 먼저 담으면 됩니다..

• set 함수로 결과 데이터를 담아 전달

```
Button(onClick = {  
    //이전 화면으로 되돌아가면서 데이터 전달  
    navController.previousBackStackEntry?.savedStateHandle?.set("msg", "kim")  
    navController.popBackStack()  
}) {  
    Text("Go Back")  
}
```

NavHostController

- 받는 곳에서도 동일한 이름으로 획득하면 됩니다.

• 결과 데이터 획득

```
val msg =  
    navController.currentBackStackEntry?.savedStateHandle?.get<String>("msg")
```

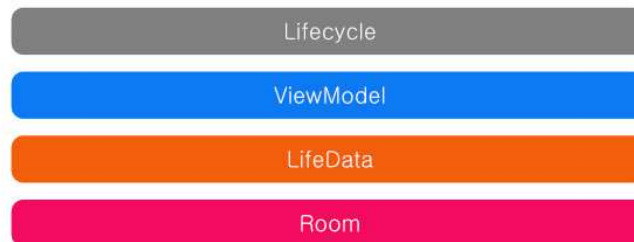
03

ViewModel

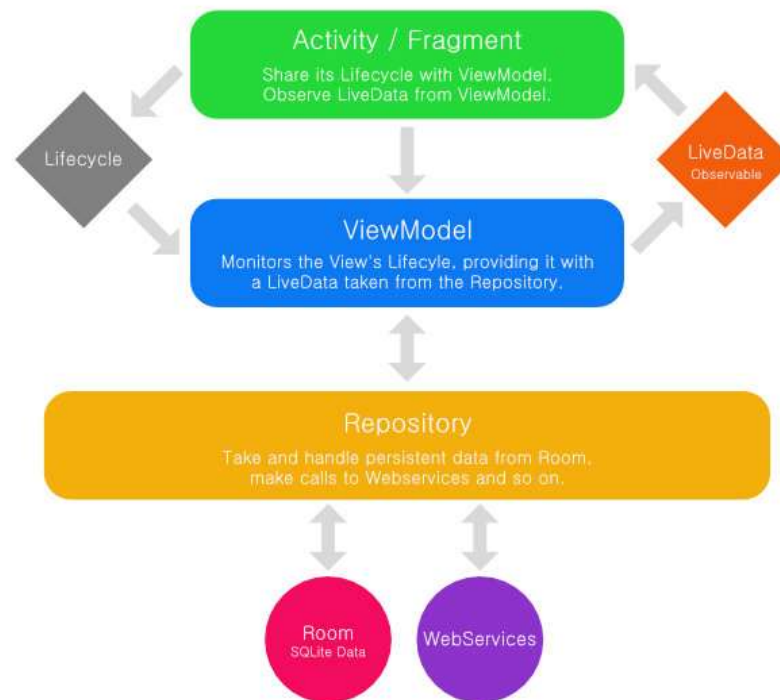
Android Architecture Component

Android Architecture Component

- 2017 Google I/O 에서 발표
- 2018 Google I/O 에서 발표된 JetPack 으로 통합
- Architecture Components 가 공개되기 전까지는 안드로이드 앱에서 특정 Architecture가 권장되지 않았다.
- MVP, MVC, MVVM, MVPP 등 다양한 아키텍처가 사용
- 안드로이드 앱을 위한 아키텍처를 정의하고 이를 구현하기 위한 라이브러리를 제공하고자 architecture components 을 공개
- Separation of concerns 을 목적으로 함.
- 그로인한 UI 와 모델이 분리



Android Architecture Component



Android Architecture Component

Activity/ Fragment : View

- UI를 구성하고 유저와 상호 작용. LifeCycle 변경을 감지.
- B/L 처리를 위해 ViewModel 이용.
- ViewModel에서 전달한 LiveData 의 내용을 화면에 출력.

ViewModel

- View와 Model을 분리시키기 위한 가교 역할.
- View의 요청에 의한 Repository 실행.
- Repository에서 발생한 데이터를 LiveData로 View에 전달.

Repository

- 데이터의 저장 및 획득이 주 목적
- Room : 데이터 영속화

ViewModel

- MVVM 의 핵심은 화면에서 B/L 을 분리해서 개발
- 컴포저블에서 UI 를 처리하고 ViewModel 에서 B/L 을 처리

```
implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.x")
```

ViewModel

- ViewModel 상속으로 작성

```
class MyViewModel1: ViewModel() {  
}
```

ViewModel

Activity 에서 ViewModel 이용

- ViewModelProvider 를 이용한 객체생성

```
val viewModel = ViewModelProvider(this).get(MyViewModel1::class.java)
```

- 직접 객체생성도 가능

```
val viewModel = MyViewModel1()
```

- ViewModelProvider 을 이용한 객체생성해야 Activity 상태 변화에 따른 ViewModel 이 재 생성되는 것을 막을 수 있다.

- Kotlin property delegate 이용

```
val viewModel: MyViewModel1 by viewModels()
```

ViewModel

컴포저블에서 ViewModel 이용

- LocalViewModelStoreOwner를 통해 현재 컴포지션의 Owner를 획득 이용

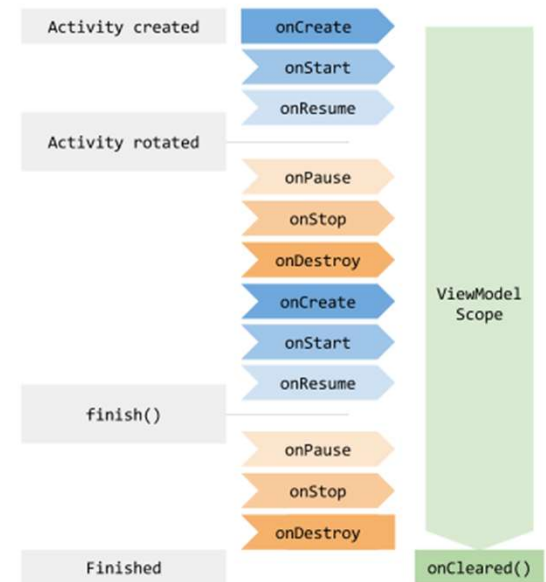
```
@Composable
fun MyScreen() {
    val owner = LocalViewModelStoreOwner.current!!
    val vm = ViewModelProvider(owner).get(MyViewModel::class.java)
}
```

- viewModel() 헬퍼 함수 제공

```
@Composable
fun MyScreen() {
    val vm: MyViewModel = viewModel() // 위와 동일한 결과
}
```

ViewModel Lifecycle

- Activity Scope 내에서 Singleton
- ViewModel 객체가 종료되는 것은 Activity가 finished 되거나 Fragment 가 detach 되었을 때



AndroidViewModel

- AndroidViewModel 은 ViewModel 의 서브 클래스

[ViewModel\(\)](#)
[AndroidViewModel\(Application application\)](#)

- Application 객체를 이용해 Context 객체 획득

```
class MyAndroidViewModel(application: Application) : AndroidViewModel(application) {  
  
    val a = application  
    val b = getApplication<Application>()  
  
    // 커스텀 Application 클래스가 있을 때만 getApplication()이 유용  
    val c = getApplication<MyApplication>()  
    val d = application as MyApplication  
}
```



03



LiveData, MutableState, StateFlow



Android Architecture Component

ViewModel 데이터 발행

- LiveData : 레거시, AAC API
- MutableState : Compose API
- StateFlow : Coroutine Flow

레거시

LiveData

androidx.lifecycle

```
// ViewModel val count =
MutableLiveData(0) // Composable val
count by viewModel.count
.observeAsState(0)
```

소속

androidx.lifecycle

추가 의존성

compose-runtime-livedata

비동기 처리

불편

초기값 null

주의 필요

XML 공용

가능

생명주기 인식

자동

권장 여부

비권장

권장

StateFlow

kotlinx.coroutines

```
// ViewModel val count =
MutableStateFlow(0) // Composable
val count by viewModel.count
.collectAsState WithLifecycle()
```

소속

kotlinx.coroutines

추가 의존성

lifecycle-runtime-compose

비동기 처리

자연스러움

초기값 null

항상 존재

XML 공용

가능

생명주기 인식

WithLifecycle

권장 여부

표준 권장

제한적 사용

MutableState

androidx.compose.runtime

```
// ViewModel var count by
mutableStateOf(0) private set //
Composable // 그대로 읽기
viewModel.count
```

소속

compose.runtime

추가 의존성

없음

비동기 처리

불편

초기값 null

항상 존재

XML 공용

불가

생명주기 인식

없음

권장 여부

간단할 때만

LiveData

- ViewModel 에서 LiveData 로 데이터 발행

```
class MyViewModel: ViewModel() {  
  
    fun someData() : MutableLiveData<String> {  
        val liveData= MutableLiveData<String>()  
        thread {  
            SystemClock.sleep(3000)  
            liveData.postValue("world")  
        }  
        return liveData  
    }  
}
```

LiveData

- LiveData의 변경을 감지하는 observer는 Observer 를 구현한 클래스
- LiveData 의 값이 변경되면 Observer 의 onChanged() 함수가 자동 호출

```
val observer = object : Observer<String> {  
    override fun onChanged(t: String?) {  
        Log.d("kkang", "onChanged.....$t")  
    }  
}  
model.someData2().observe(this, observer)
```

- observeAsState() : Observer + State

```
val count by viewModel.count.observeAsState(0)
```

MutableState

- MutableState는 Compose의 상태이기 때문에, 값이 변경되면 해당 상태를 읽고 있는 컴포저블에 자동으로 recomposition이 발생
- 상태를 실제로 읽은 컴포저블만 recomposition

```
class MyViewModel : ViewModel() {  
    var count by mutableStateOf(0)  
    private set  
  
    fun increment() { count++ }  
}
```

```
@Composable  
fun MyScreen(viewModel: MyViewModel = viewModel()) {  
    Text("Count: ${viewModel.count}") // 읽기  
}
```

StateFlow

- 코루틴 Flow 를 이용해 데이터 발생

```
class MyViewModel(private val repo: UserRepository) : ViewModel() {  
  
    // DB나 네트워크에서 올라오는 Flow를 그대로 노출  
    val users: StateFlow<List<User>> = repo.getUsers()  
        .stateIn(  
            scope = viewModelScope,  
            started = SharingStarted.WhileSubscribed(5000),  
            initialValue = emptyList()  
        )  
}
```

StateFlow

Flow 데이터 구독

- collect { } : 코루틴 Flow API
- collectAsState()
Compose 에서 추가한 Flow 확장함수
Flow 데이터를 구독하면서 결과를 State<T>로 변환하여 값이 바뀌면 ReComposition 트리거

```
val count by viewModel.count.collectAsState()
```


StateFlow

Flow 데이터 구독

- collectAsStateWithLifecycle()
- Flow 데이터를 State<T>로 변환 + 생명주기 인식(앱이 백그라운드로 가면 수집 중단)

```
val count by viewModel.count.collectAsStateWithLifecycle()
```

03

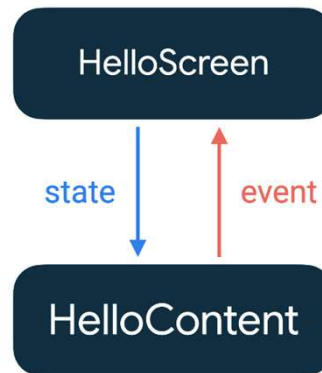
상태 호이스팅과 ViewModel

Compose

상태 호이스팅

State Hoisting

- 호이스팅은 무언가를 끌어 올린다는 의미의 단어이다.
- 어떤 컴포저블에서 상태를 변경하고자 할 때 그 컴포저블은 Stateful 로 만들어야 한다.
- 그런데 이 상태 변경을 직접 하지 않고 조상에 위임시켜(상위의 함수를 호출하는 등) 자신이 직접 상태를 관리하지 않는 것을 State Hoisting 이라고 한다.



상태 호이스팅

단일 컴포저블 상태

- 상태를 항상 호이스팅 할 필요는 없다.
- 상태를 제어하거나 이용하는 다른 컴포저블이 없다면 상태를 이용하는 해당 컴포저블내에 선언하고 관리한다.
- 또한 상태를 위한 로직이 앱의 업무와 관련없이 간단한 화면 제어만을 목적으로 한다면 ViewModel 등에 작성할 필요는 없다. 컴포저블 내에 상태와 그 상태를 위한 로직이 구현 될 수 있다.

```
@Composable
fun MyScreen() {
    var count by remember { mutableStateOf(0) }
}
```

```
@Composable
fun MyScreen() {
    var count by rememberSaveable { mutableStateOf(0) }
}
```

상태 호이스팅

부모 자식간 상태 공유

- 컴포저블 계층구조에서 동일 부모의 자식들간 상태 공유라면 특별한 기법을 사용하지 않고 부모 컴포저블에서 상태를 관리하고 자식 컴포저블에서 부모의 상태를 이용하게 구현한다.

• 상위 컴포저블의 상태 변경

```
@Composable
fun MainScreen() {
    var switchState by remember {
        mutableStateOf(true)
    }
    val onSwitchChange = { value: Boolean -> switchState = value}
    Row {
        MySwitch(switchState = switchState, onSwitchChange = onSwitchChange)
        MySwitchText(switchState = switchState)
    }
}
```

```
@Composable
fun MySwitch(switchState: Boolean, onSwitchChange: (Boolean) -> Unit){
    Switch(checked = switchState, onCheckedChange = onSwitchChange)
}

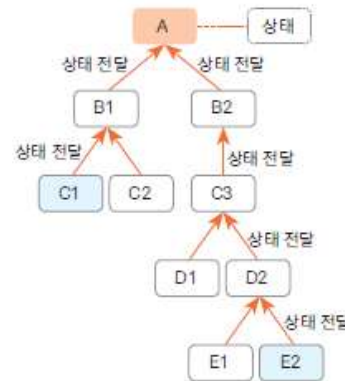
@Composable
fun MySwitchText(switchState: Boolean) {
    Text(when(switchState) {
        true -> "switch on"
        else -> "switch off"
    }, fontSize = 40.sp)
}
```



상태 호이스팅

단일 계층내에서 상태 공유(prop drilling 혹은 deep drilling)

- 여러 컴포저블 함수에서 필요한 상태를 공통의 조상 함수(상위 함수)에 선언하고 하위 함수인 컴포저블 함수를 호출하면서 상태를 매개변수로 전달하는 방식은 복잡한 경우에는 계속 매개변수로 상태 데이터를 전달한다는 것이 프로그램의 효율성 측면에서 문제가 있다.



상태 호이스팅

단일 계층내에서 상태 공유

- ProvidableCompositionLocal 객체 이용
 - ProvidableCompositionLocal을 이용하면 상위 함수의 상태 데이터를 하위 함수에 매개변수로 일일이 전달하지 않아도 됩니다.
 - compositionLocalOf() 혹은 staticCompositionLocalOf() 함수를 이용해 ProvidableCompositionLocal을 선언
 - compositionLocalOf()와 staticCompositionLocalOf()의 차이는 상태가 변경될 때 재구성 되는 컴포저블 함수에 있습니다.
 - staticCompositionLocalOf()는 상태가 변경되면 그 상태를 이용한 하위 함수부터 모든 하위 함수가 재구성됩니다.
 - compositionLocalOf()는 상태를 이용한 컴포저블 함수만 재구성합니다.

• ProvidableCompositionLocal 선언

```
val LocalProvider: ProvidableCompositionLocal<Int> = compositionLocalOf {0}  
val StaticLocalProvider: ProvidableCompositionLocal<Int> = staticCompositionLocalOf {0}
```

상태 호이스팅

단일 계층내에서 상태 공유

- CompositionLocalProvider
 - 상태를 하위 컴포저블에 공개

• ProvidableCompositionLocal로 상태 공개 예

```
CompositionLocalProvider(LocalProvider provides count) {  
    LocalChild()  
}
```

- 하위 컴포저블에서 상태를 이용할 때는 ProvidableCompositionLocal 객체의 current 프로퍼티를 사용하면 됩니다.

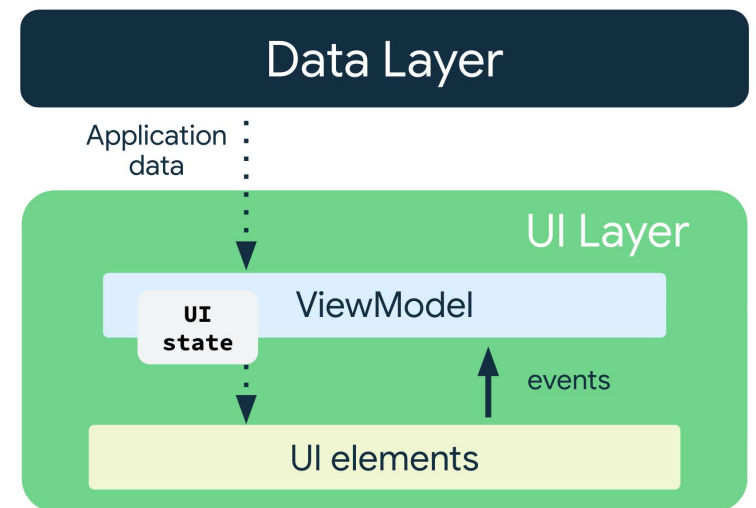
• ProvidableCompositionLocal 객체의 프로퍼티 사용 예

```
@Composable  
fun LocalChildChild() {  
    Text("local child child : ${LocalProvider.current}")  
}
```


ViewModel 과 상태

UDF(단방향 데이터 흐름)

- ViewModel은 UI가 사용하는 상태를 보유하고 노출합니다.
- UI 상태는 ViewModel에 의해 변환된 애플리케이션 데이터입니다.
- UI가 ViewModel에 사용자 이벤트를 알립니다.
- ViewModel이 사용자 동작을 처리하고 상태를 업데이트합니다.
- 업데이트된 상태가 렌더링할 UI에 다시 제공됩니다.
- 상태 변경을 야기하는 모든 이벤트에 대해 이 프로세스가 반복됩니다.



ViewModel 과 상태

ViewModel 호이스팅

- 컴포저블에 관련된 여러 UI 상태가 있고, 그 UI 상태를 위한 여러 로직이 필요한 경우 이를 분리하기 위한 홀더 클래스를 만들어 사용할 수 있다.
- 업무 상태와 업무 로직은 ViewModel 로 호이스팅
- ViewModel 이 Composable 과 동일 라이프사이클이 아님으로 무조건 여러 컴포저블을 위한 상태를 ViewModel 에 담는 것은 좋지 않다.
- 메모리 누수가 발생할 수 있다.

ViewModel 과 상태

단일 화면에서 상태 공유

- 단일 화면 내의 모든 컴포지블이 ViewModel 을 공유해야 하는 경우 viewModel(navEntry).

```
@Composable
fun MyScreen(
    navController: NavController
) {
    val backStackEntry = navController.currentBackStackEntryAsState()
    val viewModel: MyViewModel = viewModel(backStackEntry.value!!)
    // 해당 화면(route)이 백스택에서 제거될 때 ViewModel도 소멸
}
```

ViewModel 과 상태

단일 화면에서 상태 공유

- Navigation Graph 에서 ViewModel 등록

```
NavHost(navController, startDestination = "home") {  
    composable("home") {  
        val viewModel: HomeViewModel = viewModel(it) // it = BackStackEntry  
        HomeScreen(viewModel)  
    }  
    composable("detail/{id}") {  
        val viewModel: DetailViewModel = viewModel(it)  
        DetailScreen(viewModel)  
    }  
}
```

ViewModel 과 상태

특정 화면들에서 상태 공유

- Navigation Graph 에서 ViewModel 등록

```
NavHost(navController, startDestination = "main") {  
  
    // 이 nested graph 안에서만 CartViewModel 공유  
    navigation(startDestination = "cart", route = "shopping") {  
        composable("cart") {  
            val entry = navController.getBackStackEntry("shopping")  
            val viewModel: CartViewModel = viewModel(entry)  
            CartScreen(viewModel)  
        }  
        composable("checkout") {  
            val entry = navController.getBackStackEntry("shopping")  
            val viewModel: CartViewModel = viewModel(entry) // 동일 인스턴스  
            CheckoutScreen(viewModel)  
        }  
        // "shopping" 그래프를 벗어나면 CartViewModel 소멸  
    }  
    composable("home") {  
        // CartViewModel 접근 불가  
    }  
}
```

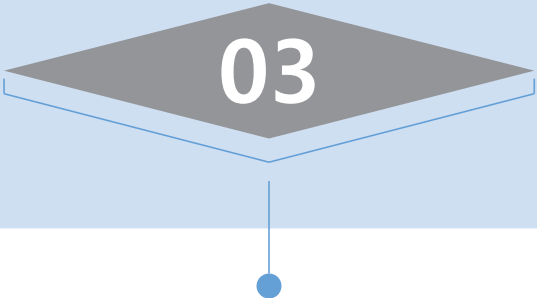
ViewModel 과 상태

앱 전체 상태 공유

- viewModel()

```
@Composable
fun HomeScreen(vm: MyViewModel = viewModel()) {
    Text("HomeScreen hashCode: ${vm.hashCode()}")
}

@Composable
fun OneScreen(vm: MyViewModel = viewModel()) {
    Text("OneScreen hashCode: ${vm.hashCode()}") // 동일한 hashCode 출력
}
```



03



Composable Lifecycle

Android Architecture Component

Lifecycle Aware Components

- Activity/ Fragment 의 Lifecycle 처리를 따로 구성
- Activity는 Lifecycle Owner 가 되며 이를 처리하는 곳이 Lifecycle Observer
- Activity 의 모든 라이프사이클 코드를 분리하겠다는 개념이 아님
- 라이프사이클과 관련된 의미단위의 작업을 추상화 시켜서 작성
- Switching between coarse and fine-grained location updates
- Stopping and starting video buffering
- Starting and stopping network connectivity
- Pausing and resuming animated drawables

Lifecycle Aware Components

- Activity/ Fragment Lifecycle 변경 시 실행될 코드 등록
- DefaultLifecycleObserver 구현한 클래스
- Fragment 의 경우 onCreateView, onAttach 등의 Fragment 만을 위한 라이프사이클 감지는 안됨

```
class MyActivityLifecycleObserver: DefaultLifecycleObserver {  
    override fun onStart(owner: LifecycleOwner) {  
        super.onStart(owner)  
    }  
  
    override fun onStop(owner: LifecycleOwner) {  
        super.onStop(owner)  
    }  
}
```

- Fragment, AppCompatActivity 에서 addObserver() 함수를 이용해 Observer 등록

```
lifecycle.addObserver(lifecycleObserver)
```

Composable Lifecycle

- 뷰를 이용해 화면을 구성하는 경우 Activity, Fragment 중심이며 Activity, Fragment 의 라이프사이클을 감지 필요
- 컴포즈를 이용해 화면을 구성하면 Activity 는 하나 필요하지만 대부분의 화면 단위가 컴포저블임으로 Activity, Fragment의 라이프사이클 감지 필요성이 줄어든다.
- StateFlow + collectAsStateWithLifecycle : 앱이 백그라운드/포그라운드 상황에 진입하는 것을 자동인지
- LaunchedEffect/ DisposableEffect : 컴포저블에서 라이프사이클 제어

LaunchedEffect

컴포저블을 위한 어떤 로직이 불필요하게 반복될 필요가 없을 때 사용

최초 한번 실행

```
LaunchedEffect(Unit) {  
    // Composition 진입 시 1회만 실행  
    viewModel.loadInitialData()  
}
```

key 변경시마다 실행

- 컴포저블이 Recomposition 된다고 하더라도 key 가 변경되지 않으면 다시 실행되지 않는다.

```
LaunchedEffect(userId, filter) {  
    // userId 또는 filter 중 하나라도 바뀌면 재실행  
    viewModel.loadFilteredData(userId, filter)  
}
```

LaunchedEffect

- LaunchedEffect { } 의 블록은 자동으로 코루틴(coroutine) 안에서 실행
- LaunchedEffect는 내부적으로 CoroutineScope.launch { } 를 사용하여 블록을 실행

```
LaunchedEffect(key1 = someKey) {  
    // 이 블록은 자동으로 코루틴 스코프 안에서 실행됨  
    delay(1000)           // suspend 함수 호출 가능  
    fetchData()          // suspend 함수 호출 가능  
    withContext(Dispatchers.IO) { ... } // 가능  
}
```

- 생명주기 연동: Composable이 컴포지션에 진입할 때 코루틴이 시작되고, 컴포지션을 벗어나면 자동으로 취소(cancel)
- 키(key) 변경 시 재실행: key 값이 바뀌면 기존 코루틴을 취소하고 새 코루틴을 시작.

LaunchedEffect

- 기본적으로 Dispatchers.Main 에서 실행
- withContext를 사용해 블록 내부에서 Dispatcher를 전환

```
LaunchedEffect(key1) {  
    // 기본: Dispatchers.Main  
  
    withContext(Dispatchers.IO) {  
        // IO 작업 (네트워크, DB)  
        val data = repository.fetchData()  
    }  
  
    withContext(Dispatchers.Default) {  
        // CPU 집약적 작업  
        val result = heavyComputation()  
    }  
  
    // 다시 Main으로 돌아옴  
    uiState = result  
}
```

DisposableEffect

- LaunchedEffect 와 다르게 onDispose { } 를 가진다.

```
DisposableEffect(key) {  
    val listener = registerSomething()  
  
    onDispose { // ← 필수! 없으면 컴파일 에러  
        listener.unregister()  
    }  
}
```

- onDispose { } 부분이 실행되는 시점
Composition에서 제거 시
key 변경으로 재실행 직전
- 코루틴에 의해 실행되지 않는다.
DisposableEffect의 목적 자체가 "등록 → 해제"의 쌍을 보장하는 것이기 때문

SideEffect

- Recomposition이 성공적으로 완료된 후마다 실행
- 키 조건 없다. 매번 실행된다.
- 동기실행이다.

```
SideEffect {  
    println("SideEffect 실행") // count 바뀔 때마다 계속 출력  
}
```

	LaunchedEffect	DisposableEffect	SideEffect
실행 방식	코루틴	동기	동기
suspend 호출	✓	✗	✗
onDispose	✗	✓ (필수)	✗
주 용도	비동기 작업	리소스 등록/해제	외부 상태 동기화

Composition vs Recomposition

- Composition 은 컴포저블이 화면에 최초로 나오거나 화면 이동했다가 다시 나오는 순간
- Recomposition 은 컴포저블의 상태가 변경되어 UI 구성을 다시 해야 하는 순간
-
- 화면전환시 백스택에 유지되는 정보는 컴포저블 자체가 아니며 컴포저블에서 이용한 뷰모델, rememberSavable 로 유지한 상태데이터이다.

Composition vs Recomposition

```
@Composable
fun A(){
    LaunchedEffect {
        //1
    }
    DisposableEffect {
        //2
        onDispose{
            //3
        }
    }
    SideEffect {
        //4
    }
    //5
}
```

상황	1 (LaunchedEffect)	2 (DisposableEffect 본문)	3 (onDispose)	4 (SideEffect)	5 (Composable 본문)
최초 화면 진입	✓	✓	✗	✓	✓
A → B 이동	✗	✗	✓	✗	✗
B → A 복귀	✓	✓	✗	✓	✓
A 제거	✗	✗	✓	✗	✗
remember 상태 변경	✗	✗	✗	✓	✓
rememberSaveable 상태 변경	✗	✗	✗	✓	✓

03

Room

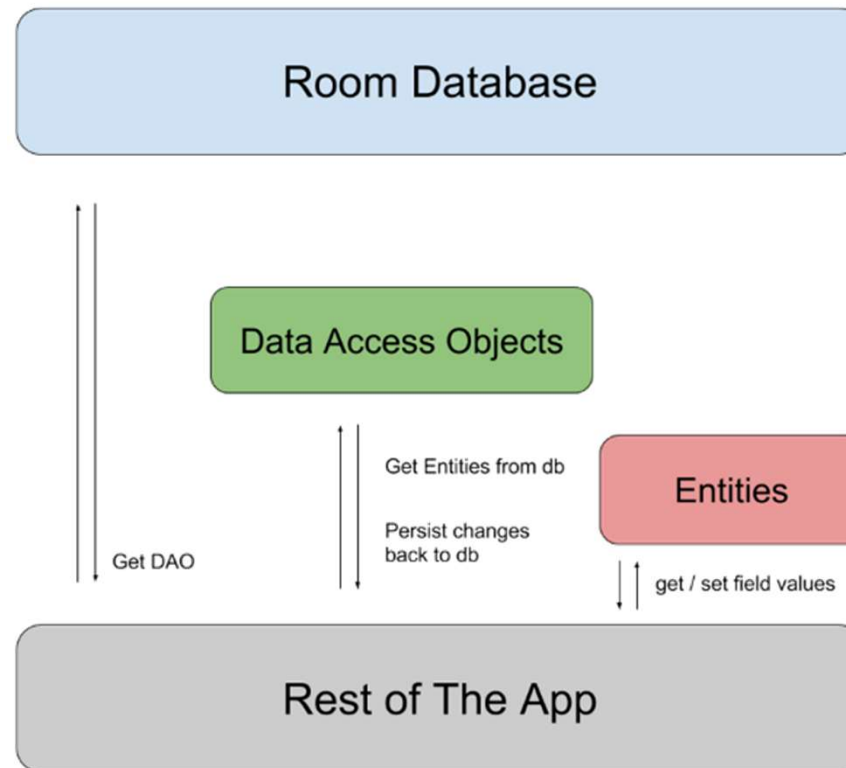
Android Architecture Component

Room

- SQLite 추상화 라이브러리
- 안드로이드 앱의 데이터베이스 이용을 Room 으로 권고

```
implementation 'androidx.room:room-ktx:2.3.0 '  
ksp("androidx.room:room-compiler:2.3.0")
```

Room



Entity

- 데이터 구조를 표현하기 위한 클래스
- @Entity 어노테이션으로 정의
- 클래스 내에 @PrimaryKey, @ColumnInfo 등의 어노테이션으로 변수 선언

```
@Entity
class User(
    @field:PrimaryKey
    var uid: Int,

    @field:ColumnInfo(name = "first_name")
    var firstName: String?,

    @field:ColumnInfo(name = "last_name")
    var lastName: String?
)
```

DAO

- DBMS를 위해 호출되는 함수를 선언하는 인터페이스
- @DAO 어노테이션으로 선언
- @Query, @Insert, @Delete 등의 어노테이션으로 함수 선언

```
@Dao
interface UserDao {
    @Query("SELECT * FROM User")
    fun getAll(): List<User>

    @Insert
    fun insertUser(uses: User)
}
```

Database

- 데이터베이스 이용을 위한 accessor(함수)를 제공
- @Database 어노테이션으로 선언
- 추상클래스로 작성
- Entity 를 어노테이션 매개변수로 지정
- 추상 함수를 가져야 하며 이 함수가 데이터베이스를 이용하기 위해 호출되는 함수
- 추상 함수는 매개변수가 없어야 하며 return 은 DAO 클래스여야 함

```
@Database(entities = arrayOf(User::class), version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
```

using Room

- Activity에서 DAO 클래스의 이용은 background 스레드로 실행
- RoomDatabase의 객체는 가급적 singleton 으로 획득할 것을 권장

```
val db = Room.databaseBuilder(  
    applicationContext,  
    AppDatabase::class.java, "test6"  
).build()
```

```
dao = db.userDao()  
  
var user = User(0, firstName = "gildong", lastName = "hong")  
thread {  
    dao.insertUser(user)  
}
```


Entity

- Entity Class
 - Entity 클래스는 @Entity 로 선언되는 클래스
 - Entity 클래스에 데이터를 저장하기 위한 table 이 자동으로 만들어 짐
 - table 이름은 클래스명이 되며, 대소문자는 무시됨
 - @PrimaryKey 어노테이션으로 PK property 를 지정할 수 있음
 - 생성자는 자유롭게 추가 가능

Entity

•Entity property

- Entity 클래스내의 property 에 해당되는 column 이 만들어 짐
- property 명에 해당되는 column 과 매핑이 되며 대소문자를 구분
- property 중 데이터베이스와 상관없이 소스적으로만 이용하고자 하는 field 는 @Ignore 어노테이션을 추가하면 됨
- 데이터베이스와 매핑되는 property 는 생성자 property 혹은 class body 에 정의 가능
- 생성자 property 에는 @Ignore 추가할 수 없음
- 자바로 변형 시 field 는 public 으로 선언되거나 private 으로 선언되면 getter/setter 함수가 추가되어 있어야 함

```
@Entity
class User2(
    @PrimaryKey(autoGenerate = true) var id: Int=0,
    var lastName: String,
    var firstName: String,
    var address: String ="seoul"
)
```

Entity

- @PrimaryKey

- Entity 클래스에는 최소 하나 이상의 primary key field 가 선언되어야 함
- autoGenerate=true 속성으로 값이 자동 증가되게 만들 수 있음
- autoGenerate=true 가 선언되었다고 하더라도 실제 데이터가 지정이 되면 그 값으로 DB INSERT 가 됨
- 만약 id 값이 0이 설정되면 값이 자동 증가
- 자동으로 insert 된 id 값은 max id 값에 1이 더해진 값

```
@PrimaryKey(autoGenerate = true) var id: Int=0,
```

- 여러 컬럼을 묶어서 primary key 로 선언할 수 있음
- primaryKeys 에 나열된 property 는 non-nullable 로 선언되어 있어야 함

```
@Entity(primaryKeys = arrayOf("lastName", "firstName"))
```

Entity

- @Entity
 - 테이블명은 기본으로 Entity 클래스명을 따르지만 원한다면 바꿀 수 있음
 - @Entity 어노테이션에 tableName 속성으로 table 이름을 지정
- @ColumnInfo
 - @ColumnInfo(name="first_name") 를 이용하여 field명과 다른 column명 지정 가능
- index
 - indices 속성으로 @Entity에 index 정보 설정

```
@Entity(  
    tableName = "tb_user",  
    indices = arrayOf(  
        Index(value = ["lastName"]),  
        Index(value = ["firstName", "lastName"], unique = true)  
    )  
)
```

foreignKey

•onDelete 속성 혹은 onUpdate을 이용해 parent table 의 데이터가 수정,삭제 되었을 때 어떻게 되어야 하는지를 명시

- CASCADE : 모든 child row를 삭제, 수정
- NO_ACTION : child 쪽에서는 아무 일도 발생하지 않음
- RESTRICT : 만약 child에 parent key 로 매핑 된 데이터가 있다면 parent 부분의 삭제 및 수정을 금지
- SET_DEFAULT : child key column 을 default 값으로 셋팅
- SET_NULL : child key column 을 null 로 셋팅

```
@Entity(  
    foreignKeys = arrayOf(  
        ForeignKey(  
            entity = User::class,  
            parentColumns = arrayOf("id"),  
            childColumns = arrayOf("user_id"),  
            onDelete = ForeignKey.CASCADE,  
        )  
    )  
)
```

DAO

- 데이터를 저장하거나 획득하기 위한 작업 클래스
- DAO 클래스는 interface 혹은 추상 클래스로 작성하며 추상함수에 정의
- 어노테이션 정보를 보고 DB 작업을 하는 클래스를 generate
- DAO 의 함수 호출은 background thread 에서 실행
- Main Thread 에서 DAO 를 실행하고자 한다면 Database Builder에서 allowMainThreadQueries() 를 호출
- DAO 의 반환 값은 LiveData 혹은 RxJava 의 Flowable 이 될 수 있음

@Insert

- insert 함수의 매개변수는 insert되는 entity 객체 하나 일수도 있고, 배열이거나 List 일 수도 있음
- @insert 함수는 insert 되는 row 의 pk 값을 받을 수도 있음
- 하나만 insert 시킨다면 insert 된 row 의 pk 가 long 타입으로 리턴
- 배열이나 List 인 경우에는 long[] 혹은 List<Long> 타입으로 리턴

```
@Insert
```

```
fun insertUser(uses: User)
```

```
@Insert
```

```
fun insertUsers(vararg users: User): List<Long>
```

```
@Insert
```

```
fun insertBothUsers(user1: User, user2: User)
```

```
@Insert
```

```
fun insertUsersAndFriends(users: List<User>)
```

@Update, @Delete

- @Update
 - 매개변수로 대입된 객체의 데이터를 모두 업데이트
 - 리턴 값은 업데이트된 row count 값을 Int 로 받을 수 있음

```
@Update  
fun updateUsers(vararg users: User)
```

- @Delete
 - 대입된 객체와 PK 를 기준으로 row 를 삭제
 - 리턴 값은 삭제된 row count 값을 Int 로 받을 수 있음

```
@Delete  
fun deleteUsers(vararg users: User)
```


@Query

- @Query 은 모든 어노테이션의 기본
- select 만을 목적으로 하지 않고 read/write 가 가능함
- sql 문은 직접 작성이 가능함
- @Query 에 정의된 sql 문은 컴파일 단계에서 검증이 이루어 짐
- parameter 설정은 콜론(:) 기호를 이용함

```
@Query("SELECT * FROM User")  
fun getAll(): List<User>
```

```
@Query("UPDATE User SET firstName = :value WHERE id = :id")  
fun updateName(value: String?, id: Int): Int
```

@Query

- Entity 클래스로 결과값을 받지 않고 select 된 일부의 데이터만 매핑되는 POJO 클래스 사용 가능

```
class NameVO {  
    @ColumnInfo(name = "firstName")  
    var fName: String? = null  
    var lastName: String? = null  
}
```

```
@Query("SELECT firstName, lastName FROM User")  
fun loadFullName(): List<NameVO>
```

@Query

- DAO function 을 suspend 함수로 선언 가능

```
@Query("SELECT * FROM user WHERE id = :id")  
suspend fun getUserSuspendFunction(id: Int): User
```

```
launch {  
    dao.getUserSuspendFunction(1).apply {  
        Log.d("kkang", "1, ${firstName}, ${lastName}")  
    }  
}
```

- @Query 의 결과로 LiveData 이용 가능

```
@Query("SELECT * FROM user WHERE id = :id")  
fun getUserLiveData(id: Int): LiveData<User>
```

```
dao.getUserLiveData(1).observe(this){  
    Log.d("kkang", "2, ${it.firstName}, ${it.lastName}")  
}
```

@Query

- @Query 의 결과로 Flow 이용 가능

```
@Query("SELECT * FROM user")  
fun getUserFlow(): Flow<User>
```

```
launch {  
    dao.getUserFlow().collect {  
        Log.d("kkang", "3, ${it.firstName}, ${it.lastName}" )  
    }  
}
```

- @Query 의 결과로 Cursor 이용 가능

```
@Query("SELECT * FROM user WHERE id = :id")  
fun getUserCursor(id: Int): Cursor
```

```
val cursor = dao.getUserCursor(1)  
if(cursor.moveToFirst()){  
    Log.d("kkang", "4, ${cursor.getString(1)}, ${cursor.getString(2)}" )  
}
```

Migration

- 데이터베이스에 스키마를 변경
- 스키마 변경은 Database 클래스의 버전 정보를 이용

```
val migration = object: Migration(1,2){  
    override fun migrate(database: SupportSQLiteDatabase) {  
        database.execSQL("ALTER TABLE USER ADD COLUMN email TEXT DEFAULT null ")  
    }  
}  
  
@Database(entities = arrayOf(User::class), version = 2)  
abstract class AppDatabase : RoomDatabase() {  
    abstract fun userDao(): UserDao  
}
```

```
val db = Room.databaseBuilder(  
    applicationContext,  
    AppDatabase::class.java, "test6"  
)  
    .addMigrations(migration)  
    .build()
```

Converter

- 기초 타입 이외의 타입 매핑은 지원하지 않음

```
@Entity
class User(
    @PrimaryKey(autoGenerate = true)
    var id: Int=0,
    var lastName: String,
    var datas: List<String>
)
//Cannot figure out how to save this field into database. You can consider adding a type converter for it.
//private java.util.List<java.lang.String> datas;
```

Converter

- @TypeConverter 로 어떻게 데이터가 매핑 되어야 하는지를 명시해서 사용

```
class Converters {  
    @TypeConverter  
    fun fromListToString(value: List<String>?): String {  
        return Gson().toJson(value)  
    }  
  
    @TypeConverter  
    fun fromStringToList(value: String): List<String> {  
        return Gson().fromJson(value, Array<String>::class.java).toList()  
    }  
}
```

```
@Database(entities = arrayOf(User::class), version = 2)  
@TypeConverters(Converters::class)  
abstract class AppDatabase : RoomDatabase() {  
    abstract fun userDao(): UserDao  
}
```

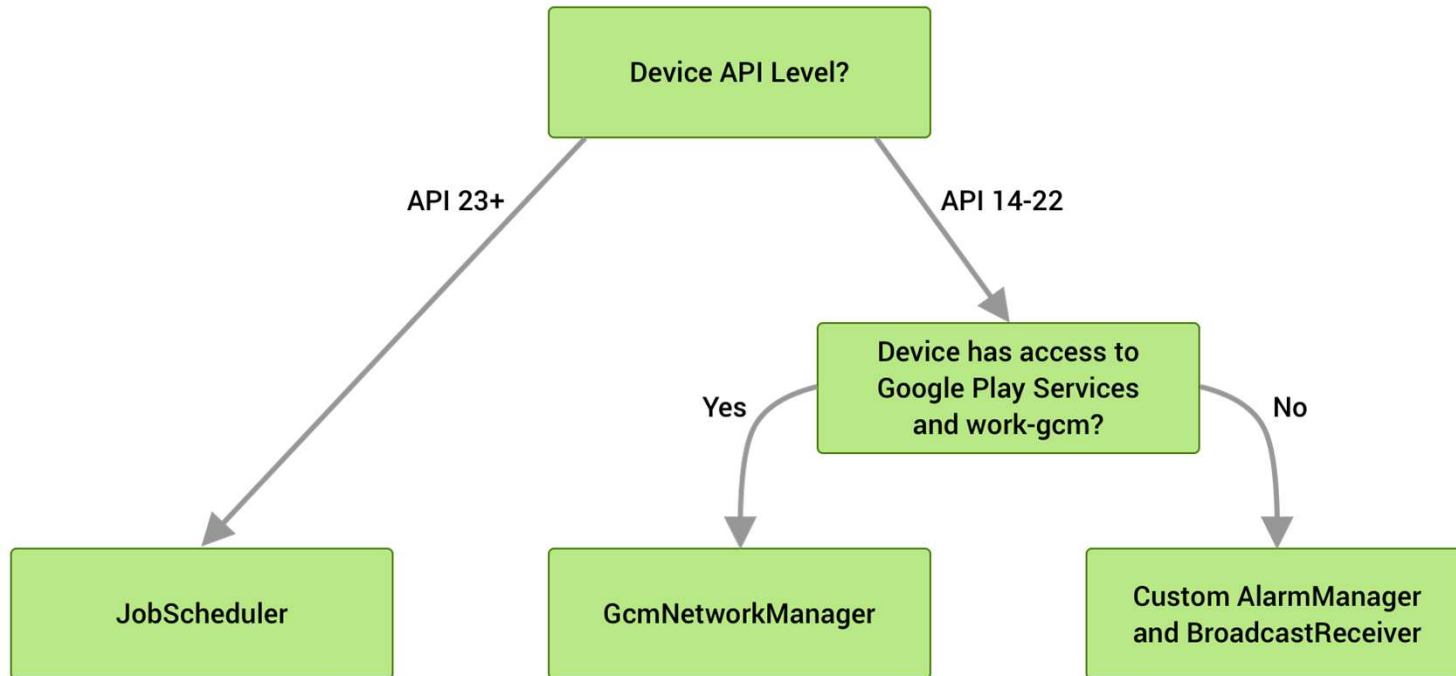
03

WorkManager

Android Architecture Component

WorkerManager

- WorkManager는 특정 작업을 유예(Defer) 처리, 비동기 처리를 제공
- JobScheduler, FirebaseJobDispatcher, AlarmManager 등을 통합한 라이브러리



WorkerManager

- 작업이 실행될 제약조건을 명시
- 한번만 실행될 작업 혹은 반복적으로 실행될 작업 등록 제공
- 작업 체이닝 제공
- 앱이 disable 상태, 앱이 종료된 상태, 기기가 다시 부팅되는 상태에서도 특정 조건에 실행 되어야할 작업

```
implementation "androidx.work:work-runtime-ktx:$work_version"
```

Worker

- 특정 상황에서 실행될 작업을 명시하는 개발자 클래스
- Worker 를 상속받아 작성
- doWork() 함수에 실행할 업무 등록

```
class MyWorker (context: Context, val workerParams: WorkerParameters) : Worker(context, workerParams) {  
    override fun doWork(): Result {  
        Log.d("kkang", "MyWorker..doWork()....." + workerParams.id + "," + workerParams.tags)  
        return Result.success()  
    }  
}
```

- WorkerParameters : id, tag, 전달된 데이터
- Result : success(), failure(), retry() 중 하나로 결과 리턴
- success(), failure() 는 작업이 종료된 것을 의미하며 retry() 는 다시 작업이 등록됨

Constraints

- WorkRequest에 등록되는 작업이 실행되는 조건
- Constraints.Builder에 의해 생성
- RequestCharging : 기기의 충전상태
- BatteryNotLow : 기기의 배터리가 부족하지 않은 상태
- DeviceIdle : 기기가 idle 상태일 때
- StorageNotLow : 기기의 저장공간이 부족하지 않을 때
- NetworkType : 기기가 WIFI 네트워킹이 가능한 상태일 때

```
val myConstraints = Constraints.Builder()  
    .setRequiredNetworkType(NetworkType.UNMETERED)  
    .build()
```

WorkRequest

- 작업에 대한 요청정보
- Worker 와 Constraints 를 등록
- OneTimeWorkRequest, PeriodicWorkRequest 두개의 클래스 제공

```
val workRequest = OneTimeWorkRequestBuilder<MyWorker>()  
    .setConstraints(myConstraints)  
    .addTag("someTask")  
    .build()
```

- WorkRequest 등록

```
val workManager = WorkManager.getInstance(this)  
workManager.enqueue(workRequest)
```

Tag

- Tag 는 WorkRequest에 추가하는 문자열 정보
- WorkRequest는 자동으로 id 값 부여되어 식별되지만 Tag 는 여러 WorkRequest 에 동일 문자열로 지정하는 것이 가능
- Tag 로 여러 request 를 group 으로 묶어 한꺼번에 취소 가능

```
val workRequest = OneTimeWorkRequestBuilder<MyWorker>()  
    .setConstraints(myConstraints)  
    .addTag("someTask")  
    .build()
```

```
workManager.cancelAllWorkByTag("someTask")
```

- id 값으로 취소 가능

```
val workId = workRequest.id  
workManager.cancelWorkById(workId)
```

Data – Worker 에 데이터 전달

- Worker 등록 할 때 데이터 전달
- Data 객체에 key-value 형태로 전달

```
val myData = Data.Builder()
    .putInt("data1", 10)
    .putString("data2", "hello")
    .putBoolean("data3", true)
    .build()

val workRequest = OneTimeWorkRequestBuilder<MyWorker>()
    .setInputData(myData)
    .build()
```

- Worker 의 inputData property 로 데이터 획득

```
val x = inputData.getInt("data1", 0)
```

Data – Worker 의 실행 결과 획득

- Worker 의 실행 결과를 전달
- Data 객체를 Result 에 추가해서 전달

```
val output = Data.Builder().putInt("reData", 100).build()
//or
val output: Data = workDataOf(KEY_RESULT to result)

return Result.success(output)
```

- WorkStatus 를 통해 데이터 획득

```
workManager.getWorkInfoByIdLiveData(workRequest.id)
    .observe(this, { workStatus ->
        if (workStatus != null && workStatus.getState().isFinished()) {
            val myResult = workStatus.outputData.getInt("reData", 0)

        }
    })
```


Progress

- Worker 의 진행률 등록
- CoroutineWorker 의 setProgress() 로 진행률 등록

```
class MyWorker (context: Context, val workerParams: WorkerParameters) : CoroutineWorker(context, workerParams) {  
  
    override suspend fun doWork(): Result {  
        setProgress(workDataOf("myProgress" to 0))  
        //.....  
        setProgress(workDataOf("myProgress" to 100))  
        return Result.success()  
    }  
}
```

- Progress 값 획득

```
workManager.getWorkInfoByIdLiveData(workRequest.id)  
    .observe(this, { workStatus ->  
  
        val progress = workStatus.progress  
        val value = progress.getInt("myProgress", 0)  
  
    })
```

Chaining Work

- 여러 작업의 순서 정의
- WorkManager 의 beginWith() 와 then() 함수를 이용하여 작업의 순서를 지정
- 복잡한 작업의 Chaining 을 위해 WorkContinuation 제공

•예) workA -> workB -> workC 순으로 작업이 진행

```
workManager
    .beginWith(workA)
    .then(workB)
    .then(workC)
    .enqueue()
```

•예) workA, workB 동시 진행 -> workC -> workD, workE 동시 진행

```
workManager
    .beginWith(listOf(workA, workB))
    .then(workC)
    .then(listOf(workD, workE))
    .enqueue()
```

Chaining Work

- 복잡한 조건은 WorkContinuation
- 예) A->B 가 진행이 되고 이와 별도로 C->D 가 진행되어야 하며 B 와 D 가 모두 끝난 후 E가 실행되어야 하는 상황

```
val chain1 = workManager
    .beginWith(workA)
    .then(workB)
val chain2 = workManager
    .beginWith(workC)
    .then(workD)
val chain3 = WorkContinuation
    .combine(listOf(chain1, chain2))
    .then(workE)

chain3.enqueue()
```

03

Hilt

Android Architecture Component

Hilt

Dagger

- Dagger 는 square사에서 처음 시작했지만 Dagger2 부터는 구글에서 주도
- 자바/코틀린 범용 DI 라이브러리

Hilt

- Hilt는 구글에서 만든 Android 전용 의존성 주입(Dependency Injection, DI) 라이브러리
- Dagger를 기반
- 컴파일 타임 검증 및 코드 생성
- Dagger에서는 ActivityComponent, FragmentComponent 등을 직접 설계해야 했지만, Hilt는 이를 미리 정의해두고 안드로이드 생명주기에 맞춰 자동으로 관리

Hilt

주요 어노테이션

- **@HiltAndroidApp** : Application 클래스에 선언하며, Hilt의 코드 생성을 시작하는 시작점
- **@AndroidEntryPoint** : Activity, Fragment 등에 선언하여 Hilt가 의존성을 주입할 수 있게 한다.
- **@Inject** : 생성자나 필드에 사용하여 "이 객체를 주입해달라"고 요청
- **@Module** : 외부 라이브러리 객체처럼 생성자를 직접 수정할 수 없는 경우, 객체 생성 방법을 정의
- **@InstallIn** : 해당 모듈이 어떤 안드로이드 컴포넌트(예: ActivityComponent)에서 사용될지 지정

Hilt

Dependency

- Project 수준의 build.gradle.kts

```
plugins {  
    id("com.google.devtools.ksp") version "2.3.7" apply false  
    id("com.google.dagger.hilt.android") version "2.59.2" apply false  
}
```

- Module 수준의 build.gradle.kts

```
plugins {  
    id("com.google.devtools.ksp")  
    id("com.google.dagger.hilt.android")  
}  
dependencies {  
    implementation("com.google.dagger:hilt-android:2.59.2")  
    ksp("com.google.dagger:hilt-android-compiler:2.59.2")  
}
```

@HiltAndroidApp

- Hilt를 사용하는 모든 앱은 @HiltAndroidApp이 붙은 Application 클래스가 반드시 선언되어 한다.
- 앱이 실행되면서 의존성 객체를 등록할 Container 준비 작업이다.

```
@HiltAndroidApp  
class MySampleApplication : Application()
```


@Inject constructor()

- 주입 대상 객체 선언
- Hilt 에 의해 객체 생성 관리
- 클래스내에 @Inject constructor() 은 하나만 선언 가능하며 주생성자에 선언하는 것이 권장사항.

```
// 주입할 대상 (의존성)
class AnalyticsService @Inject constructor() {
    fun logEvent(msg: String) {
        println("Event Log: $msg")
    }
}
```

@Inject constructor()

- 매개변수를 가지는 생성자 이용 가능
- 매개변수를 Hilt 에서 관리하는 객체 Container 에서 찾아서 자동 주입
- 매개변수로 들어가는 클래스들도 Hilt가 어떻게 만드는지 알고 있어야 한다.
- 만약 매개변수가 외부 라이브러리 클래스(수정 불가능)인데 별도의 설정(Module)이 없다면 Hilt는 "이걸 어떻게 만들어야 할지 모르겠다"며 컴파일 에러

```
// Hilt가 UserRepository와 AnalyticsHelper를 컨테이너에서 꺼내와서
// MainViewModel을 생성할 때 자동으로 주입해줍니다.
class MainViewModel @Inject constructor(
    private val repository: UserRepository,
    private val analytics: AnalyticsHelper
) { ... }
```

@AndroidEntryPoint

- @HiltAndroidApp 은 앱을 위한 의존성 객체 Container 준비이며 @AndroidEntryPoint 은 액티비티, 서비스 등의 컴포넌트부터 코드가 해석되어 Container 에 있는 객체들이 주입하는 역할
- 안드로이드 시스템에서 관리하는 클래스는 직접 생성하거나 생성자를 개발자 임의로 조작할 수 없다.
- Hilt 에 의해 생성자 주입이 불가함으로 클래스 내부 변수에 직접 주입해야 한다.
- @Inject lateinit var 로 선언된 변수에 객체를 주입하는 코드가 자동 삽입되게 된다.

```
@AndroidEntryPoint
class MainActivity : AppCompatActivity() {

    // Hilt가 알아서 AnalyticsService 인스턴스를 생성해서 넣어줍니다.
    @Inject lateinit var analytics: AnalyticsService

}
```

- Activity, Fragment(프래그먼트에 추가하려면 프래그먼트를 출력하는 액티비티에도 선언되어야 한다.), 커스텀 뷰, Service, BroadcastReceiver

@HiltViewModel

- 뷰모델 선언은 @HiltViewModel 어노테이션을 이용해야 하며 생성자는 @Inject 로 선언되어야 한다.

```
@HiltViewModel
class MyViewModel @Inject constructor(
    private val repository: MyRepository
) : ViewModel() {
    // ...
}
```

@HiltViewModel

Activity/Fragment 에서 이용

- by viewModels() 을 이용하여 @HiltViewModel 로 선언된 뷰모델 이용
- 액티비티에 @AndroidEntryPoint가 붙어 있고, ViewModel에 @HiltViewModel이 붙어 있다면 Hilt가 내부적으로 ViewModelProvider.Factory를 대신 만들어서 by viewModels()에 추가

```
@AndroidEntryPoint
class StockActivity : AppCompatActivity() {
    // Hilt가 StockViewModel을 생성하고 내부 의존성까지 채워서 전달해줌
    private val viewModel: StockViewModel by viewModels()
}
```

@HiltViewModel

Compose 에서 이용

- Compose 환경에서는 hiltViewModel() 함수를 사용하여 주입

```
@Composable
fun StockScreen(
    // Hilt 컨테이너로부터 ViewModel을 자동으로 가져옴
    viewModel: StockViewModel = hiltViewModel()
) {
}
```

Scope 어노테이션

- @Inject constructor() 에 의해 생성되는 객체는 기본은 어디선가 이 객체를 요청할 때마다 생성
- 만약 특정 범위 내에서 하나만 생성되기를 원한다면 Scope 어노테이션 추가
- Hilt 내부에서 Scope 어노테이션 별로 객체 Container 를 따로 유지

어노테이션	유지되는 범위 (Scope)	설명
(없음)	요청 시마다 생성	주입할 때마다 매번 새 인스턴스를 만듦
@Singleton	앱 전체 (Application)	앱이 꺼질 때까지 단 하나의 객체 만 유지
@ActivityScoped	액티비티 생명주기	특정 액티비티 내에서만 동일 객체 유지
@ViewModelScoped	뷰모델 생명주기	특정 뷰모델 내에서만 동일 객체 유지

```
@Singleton
class AppDatabase @Inject constructor() { ... }
```

@ApplicationContext @ActivityContext

- @Inject constructor() 의 매개변수 타입이 Context 인 곳에 추가되는 어노테이션
- 데이터베이스(Room), SharedPreferences, 시스템 서비스 접근 등을 위해 Context가 필요한 경우가 많은데, 이때 어떤 Context를 써야 할지 명확히 해주는 역할

구분	@ApplicationContext	@ActivityContext
유효 기간	앱 실행 중 내내 (App Process와 동일)	해당 Activity가 살아있는 동안만
주요 사용처	DB, 네트워크, 설정값 저장 등 공용 객체	Toast 띄우기, Dialog 표시, 화면 이동 (Intent)
컴포넌트	SingletonComponent 등 상위 컴포넌트	ActivityComponent 내에서만 사용 가능

```
class MyPreferenceManager @Inject constructor(  
    @ApplicationContext private val context: Context // 여기에 주입!  
) {}
```


@Module

- Hilt가 스스로 인스턴스 생성 방법을 알 수 없는 타입은 Module로 알려줘야 한다

상황	이유	해결책
User (@Inject constructor)	Hilt가 직접 생성 가능	Module 불필요
List<User>	제네릭 타입은 생성자에 @Inject 불가	Module 필요
UserRepository (인터페이스)	인터페이스는 인스턴스 생성 불가	Module + @Binds
OkHttpClient (라이브러리)	내 코드가 아니라 @Inject 추가 불가	Module + @Provides
상위 클래스 타입으로 주입	어떤 구현체를 써야 할지 Hilt가 모름	Module + @Binds

@Module

- @Module 을 이용할때 @Provider 만 사용한다면 일반적으로 object 로 선언하고, @Binds 가 들어간다면 추상 클래스로 선언

```
// @Provides만 쓴다면 object가 가장 효율적
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {
    @Provides
    fun provideOkHttpClient(): OkHttpClient {
        return OkHttpClient.Builder().build()
    }
}
```

```
// @Binds를 쓰려면 반드시 abstract class
@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {
    @Binds
    abstract fun bindUserRepository(impl: UserRepositoryImpl): UserRepository
}
```

@Module

```
// @Binds + @Provides 혼용 시 abstract class + companion object
@Module
@InstallIn(SingletonComponent::class)
abstract class MixedModule {
    @Binds
    abstract fun bindUserRepository(impl: UserRepositoryImpl): UserRepository

    companion object {
        @Provides
        fun provideOkHttpClient(): OkHttpClient {
            return OkHttpClient.Builder().build()
        }
    }
}
```

@Module

@InstallIn

- 모듈에 추가한 클래스 객체를 어느 Container 에 등록할지 명시, 필수
- @InstallIn(SingletonComponent::class) // 앱 전체 생명주기
- @InstallIn(ActivityComponent::class) // Activity 생명주기
- @InstallIn(ViewModelComponent::class) // ViewModel 생명주기
- @InstallIn(FragmentComponent::class) // Fragment 생명주기

```
@Module
@InstallIn(SingletonComponent::class)
object AppModule { ... }
```

@Module

@InstallIn

- @InstallIn은 하나의 Module에 하나의 컴포넌트만 지정 가능
- 여러 컴포넌트에 설치하고 싶다면 Module을 분리
- 상위 컴포넌트에 설치하면 하위 컴포넌트에서도 자동으로 접근

```
SingletonComponent    // Application 범위
└─── ActivityComponent // Activity 범위
    └─── FragmentComponent // Fragment 범위
```

@Module

@Binds

- 주입되는 객체 타입을 명확하게 하기 위해 사용
- 인터페이스 타입으로 이용되는 객체, 상위 타입으로 이용되는 객체
- 주입되는 객체의 클래스에 @Inject constructor() 가 선언된 경우에 사용 가능
- 반드시 추상 함수로 선언

```
class AnalyticsServiceImpl @Inject constructor() : AnalyticsService

@Module
@InstallIn(SingletonComponent::class)
abstract class AnalyticsModule {
    @Binds
    abstract fun bindAnalyticsService(
        impl: AnalyticsServiceImpl
    ): AnalyticsService
}
```

@Module

@Provider

- @Provider 는 @Inject constructor() 가 선언되지 않은 클래스를 직접 생성해 Hilt 에 알려주는 역할

```
@Module
@InstallIn(SingletonComponent::class)
object NetworkModule {

    // Hilt야, Retrofit은 내가 알려주는 방법으로 만들어
    @Provides
    @Singleton
    fun provideRetrofit(): Retrofit {
        return Retrofit.Builder()
            .baseUrl("https://api.example.com")
            .build()
    }
}
```